arXiv:2502.05171v2 [cs.LG] 17 Feb 2025

利用潜推理扩展测试时计算：一种循环深度方法

乔纳斯·盖平 [1] 肖恩·麦克利什 [2] 尼尔·贾因 [2] 约翰·基兴鲍尔 [2] 西达尔特·辛格 [2] 布赖恩·R·巴托尔德森 [3]
Bhavya Kailkhura [3] **Abhinav Bhatele** [2] **Tom Goldstein** [2]

摘要

本文研究了一种新颖的语言模型架构，该架构通过在潜空间中隐式推理来扩展测试时计算。我们的模型通过迭代一个循环块来工作，从而在测试时展开到任意深度。这与主流推理模型通过生成更多标记来扩展计算的方式形成对比。与基于思维链的方法不同，我们的方法不需要任何专门的训练数据，可以在小上下文窗口中工作，并且能够捕捉难以用文字表示的推理类型。我们将一个概念验证模型扩展到 35 亿参数和 8000 亿标记。我们证明，所得到的模型能够在推理基准上提升其性能，有时提升幅度显著，计算负载相当于 500 亿参数。

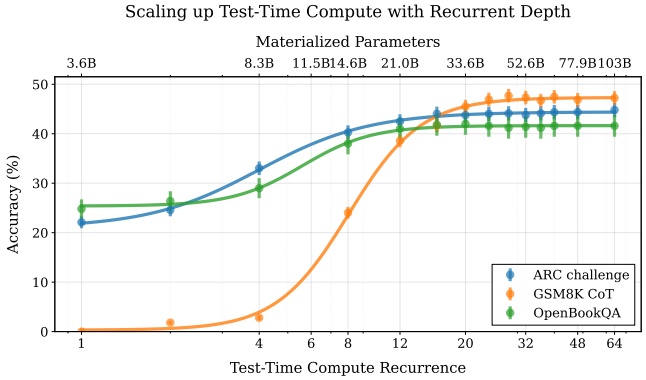


图 1：我们训练了一个具有深度循环的 35 亿参数语言模型。在测试时，模型可以迭代更长时间以使用更多计算并提升其性能。与通过长思维链“口头表达”来扩展测试时推理不同，该模型完全通过在潜空间中进行推理来改进。像 OpenBookQA 这样需要较少推理的任务比像 GSM8k 这样有效利用更多计算的任务收敛得更快。

模型： huggingface.co/tomg-group-umd/huginn-0125

代码与数据：
github.com/seal-rg/recurrentpretraining

1. 在连续空间中思考的扩展

人类在解决某些问题时自然会比其他问题投入更多的心智努力。虽然人类能够通过将中间结果口头表达并写下来，从而在长时间跨度内进行思考，但大量的思考是在说出答案的第一个词之前，通过大脑中复杂的、循环的放电模式发生的。

早期增强语言模型能力的尝试集中在扩大模型规模上，这种做法需要海量的数据和计算资源。最近，研究人员探索了通过扩展测试时计算来增强模型推理能力的方法。

[1]ELLIS Institute Tübingen, Max-Planck Institute for ~~Intelligent Systems~~, Tübingen AI Center [2] University of Maryland, College Park [3] Lawrence Livermore National Laboratory. Correspondence to: Jonas Geiping, Tom Goldstein <jonas@tue.ellis.eu, tomg@umd.edu >.

模型的能力。主流方法涉及在长思维链示例上进行后训练，以培养模型在其上下文窗口中口头表达中间计算的能力，从而将思维外化。

然而，昂贵的内部推理必须始终被压缩为单个口头表达的下一个标记，这一约束似乎是浪费的；如果模型能够在其连续的潜空间中原生地“思考”，它们可能会更有能力。解锁这一额外计算维度的未开发潜力的一种方法是为模型添加一个循环单元。该单元在循环中运行，迭代地处理和更新其隐藏状态，使计算能够无限期地进行。虽然这目前不是主导范式，但这一思想是机器学习的基础，并且在每个十年中都被（重新）发现，例如作为循环神经网络、扩散模型以及通用或循环变换器。

本文表明，深度循环语言模型能够有效学习、高效训练，并在测试时计算规模扩展下展现出显著的性能提升。我们提出的变换器架构建立在一个潜深度循环块之上，该块在训练期间以随机采样的迭代次数运行。

我们证明，该范式能够扩展到数十亿参数和超过五千亿个预训练数据标记。在测试时，模型可以通过潜空间中的循环推理提升性能，使其能够与其他受益于更多参数和训练数据的开源模型相竞争。此外，我们表明循环深度模型在推理时自然支持多种特性，而这些特性在非循环模型中需要大量的调优和研究工作，例如逐标记自适应计算、（自）推测解码以及键值缓存共享。最后，我们通过追踪潜空间中的标记轨迹完成研究，表明许多有趣的计算行为随着规模扩展自然涌现，例如模型在潜空间中旋转形状以进行数值计算。

2. 为何训练具有循环深度的模型？

循环层使变换器模型能够在发出一个标记之前执行任意多次计算。原则上，循环机制为测试时计算扩展提供了一种简单的解决方案。与更标准的长上下文推理方法相比（OpenAI，2024；DeepSeek-AI 等，2025），潜循环思维具有若干优势。

- 潜在推理无需构建定制训练数据。思维链推理要求模型在目标领域内构建的长演示上进行训练。相比之下，本文提出的潜在推理模型能够以可变计算预算进行训练，使用标准训练数据且无需专门演示，并在测试时若获得额外计算资源即可提升其能力。• 潜在推理模型在训练和推理时所需的内存少于思维链推理模型。由于后者需要极长的上下文窗口，可能需要诸如令牌并行化（Liu et al., 2023a）等专门训练方法。• 循环深度网络每参数执行的浮点运算次数多于标准变换器，显著降低了大规模加速器之间的通信成本。这尤其在互连速度较慢的情况下，能够提高设备利用率。
- 通过构建计算密集且参数规模较小的架构，本文希望为模型设定一个强先验，使其通过“思考”来解决问题，即学习元策略、逻辑和抽象，而非记忆。循环先验在学习复杂算法方面的优势已在“深度思考”文献中得到证明（Schwarzschild et al., 2021b; Bansal et al., 2022; Schwarzschild et al., 2023）。

从更哲学的角度来看，本文希望潜在推理能够捕捉人类推理中难以言表的方面，例如空间思维、物理直觉或（运动）规划。在循环过程的多次迭代中，高维向量空间中的推理将能够同时深入探索多个方向，而非线性思维，从而产生能够展现新颖且复杂推理行为的系统。

以这种方式扩展计算与通过扩展（言语化）推理扩展（Shao et al., 2024）或扩展预训练中的参数规模（Kaplan et al., 2020）并不矛盾，本文认为这可能构建了模型性能扩展的第三个轴。

目录

- 第 3 节介绍了本文的潜在循环深度模型架构和训练目标。
- 第 4 节描述了在 Frontier（AMD 集群）上进行大规模训练运行的数据选择与工程实现。
- 第 5 节报告了基准测试结果，展示了模型在扩展推理计算时的性能提升。
- 第 6 节包含若干应用示例，展示循环模型如何自然地简化大语言模型（LLM）的使用场景。
- 第 7 节可视化了在该架构与训练目标下大规模涌现的计算模式，表明上下文相关行为在潜空间中涌现，例如在响应需要数值推理的提示时出现“环绕”现象。

3. 一种可扩展的循环架构

本节将描述我们提出的具有潜在循环深度的变换器架构，讨论设计选择与小规模消融实验。架构图见图 2。我们始终将序列维度记作 n，模型的隐藏维度记作 h，词汇表记作集合 V。

3.1. 宏观设计

该模型主要围绕仅解码器的变换器块构建（Vaswani et al., 2017; Radford et al., 2019）。然而，这些块被组织为三个功能组：前奏 P，使用多个变换器层将输入数据嵌入到潜空间；核心循环块 R，是循环计算的中心单元，负责修改状态 $\mathbf{s} \in \mathbb{R}^{n \times h}$；尾声 C，使用若干层从潜空间解嵌，并包含模型的预测头。核心块位于前奏块与尾声块之间，通过循环核心块，我们可以在歌曲中放入任意数量的诗节。

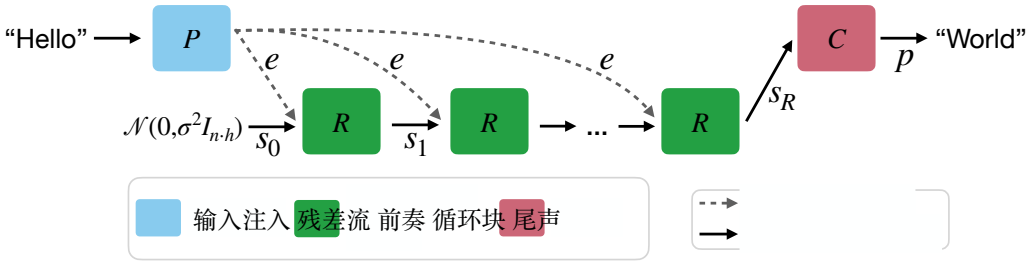


图 2：架构可视化，如第 3 节所述。每个块由若干子层组成。蓝色前奏块将输入嵌入潜空间，绿色共享循环块是一组重复计算的层，用于计算最终潜状态，该状态由红色尾声块的层进行解码。

给定若干循环迭代 r 以及输入标记序列 $\mathbf{x} \in V^n$，这些分组按以下方式用于生成输出概率 $\mathbf{p} \in \mathbb{R}^{n \times |V|}$

$$\begin{aligned} \mathbf{e} &= P(\mathbf{x}) \\ \mathbf{s}_0 &\sim \mathcal{N}(\mathbf{0}, \sigma^2 I_{n \cdot h}) \\ \mathbf{s}_i &= R(\mathbf{e}, \mathbf{s}_{i-1}) \qquad \text{对} \quad i \in \{1, \ldots, r\} \\ \mathbf{p} &= C(\mathbf{s}_r), \end{aligned}$$

其中 σ 是用于初始化随机状态的某个标准差。该过程如图 2 所示。给定初始随机状态 $\mathbf{s}_0$，模型反复应用核心模块 R，该模块接受潜在状态 $\mathbf{s}_{i-1}$ 和嵌入输入 $\mathbf{e}$，并输出新的潜在状态 $\mathbf{s}_i$。完成所有迭代后，尾声模块处理最后一个状态并生成下一个词元的概率。

该架构基于深度思考文献，其中表明在每一步注入潜在输入 $\mathbf{e}$（Bansal 等，2022）并用随机状态初始化潜在向量，可稳定递归并促进收敛到与初始化无关的稳态，即路径无关性（Anil 等，2022）。

该设计的动机。这种循环设计是学习稳定迭代算子所需的最小设置。一个很好的例子是函数 $E(\mathbf{x}, \mathbf{y})$，的梯度下降，其中 $\mathbf{x}$ 可能是感兴趣的变量，$\mathbf{y}$ 是数据。对该函数进行梯度下降时，从初始随机状态 $\mathbf{x}_0$ 开始，反复应用一个简单操作（即所优化函数的梯度），该操作依赖于前一个状态 $\mathbf{x}_k$ 和数据 $\mathbf{y}$。注意，为了实际优化我们的函数，每一步都需要使用 $\mathbf{y}$。类似地，我们在循环的每一步都反复注入数据 $\mathbf{e}$。如果仅在开始时提供 $\mathbf{e}$，例如通过 $\mathbf{s}_0 = \mathbf{e}$，那么迭代过程将不稳定 [1]，因为其解将仅依赖于其边界条件。

使用若干层嵌入输入令牌的结构 1 稳定性的含义是 R 若不依赖于 $\mathbf{e}$，则不可能成为单调算子，因此无法表示严格凸且依赖数据的函数上的梯度下降（**Bauschke** 等人，2011）

将输入映射到隐藏潜空间的做法，是基于对标准固定深度变换器（Transformer）的经验分析结果（Skean 等人，2024；Sun 等人，2024；Kaplan 等人，2024）。该系列研究表明，大型语言模型（LLM）的初始层与末端层存在显著差异，而中间层则具有可互换性与可置换性。例如，Kaplan 等人（2024）指出，标准模型在少数几层内便已将子词标记嵌入到潜空间中的单一概念上，随后模型便在这些概念上进行运算。

备注 3.1（这是扩散模型吗？）。这种迭代式架构与其他现代迭代建模范式——扩散模型（Diffusion Model）（Song 和 Ermon，2019），尤其是潜空间扩散模型（Latent Diffusion Model）（Rombach 等人，2022）——看起来会很相似。我们进行了若干消融实验，采用了与扩散模型更为相似的迭代方案，例如 $\mathbf{s}_i = R(\mathbf{e}, \mathbf{s}_{i-1}) + \mathbf{n}$，其中 $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \sigma_i I_{n \cdot h})$，但在初步实验中发现注入噪声并无帮助，这可能与我们的训练目标有关。我们还评估了 $\mathbf{s}_i = R_i(\mathbf{e}, \mathbf{s}_{i-1})$，即一个以当前步骤为输入的核心模块（Peebles 和 Xie，2023），但发现这与路径无关性相互作用不佳，导致模型无法外推。

3.2. 微观设计

在每个组内，我们大体遵循标准的变换器（Transformer）层设计。每个块包含多个层，每个层包含一个使用 RoPE（Su 等人，2021）且基数为 50000 的标准因果自注意力块，以及一个门控 SiLU 多层感知机（Shazeer，2020）。我们使用 RMSNorm（Zhang 和 Sennrich，2019）作为归一化函数。模型仅在查询和键上具有可学习的偏差，其他地方没有。为了稳定循环，我们按照以下"三明治"格式排列所有层，使用归一化层 n_i，这与（Ding 等人，2021；Team Gemma 等人，2024）中的类似策略相关但不完全相同：

$$\begin{aligned} \hat{\mathbf{x}}_l &= n_2\left(\mathbf{x}_{l-1} + \text{Attn}(n_1(\mathbf{x}_{l-1}))\right) \\ \mathbf{x}_l &= n_4\left(\hat{\mathbf{x}}_l + \text{MLP}(n_3(\hat{\mathbf{x}}_l))\right) \end{aligned}$$

在小规模下，大多数归一化策略，例如前置归一化、后置归一化等，效果几乎同样好，

我们对这些选项进行了消融实验，发现这种归一化对于在大规模下训练循环是必需的 [2] 。

给定嵌入矩阵 E 和嵌入缩放 γ ，前奏块首先将输入标记 $\mathbf{x}$ 嵌入为 $\gamma E(\mathbf{x})$ ，然后应用 l_F 个具有上述布局的前奏层。

我们的核心循环块 R 以一个适配器矩阵 $A : \mathbb{R}^{2h} \to \mathbb{R}^h$ 开始，将 $\mathbf{s}_i$ 和 $\mathbf{e}$ 的拼接映射到隐藏维度 h （ （Bansal 等人，2022）。虽然通过加法而非拼接重新引入初始嵌入特征对于较小模型同样有效，但我们发现在大规模下拼接效果最佳。然后将其输入 l_R 个变换器层。在核心块末尾，输出再次通过 RMSNorm n_c 进行缩放。

尾声包含 l_C 个层，通过 n_c 进行归一化，并使用绑定嵌入 E^T 投影到词汇表。

总之，我们可以用三元组 (l_P, l_R, l_C) 来概括该架构，它描述了每个阶段的层数，以及递归次数 r ，后者在每次前向传递中可能有所不同。我们训练了若干形状为 $(1, 4, 1)$ 、隐藏尺寸为 $h = 1024$ 的小规模模型，以及一个形状为 $(2, 4, 2)$ 、 $h = 5280$. 的大型模型。该模型仅有 8 个“真实”层，但当递归块被迭代（例如 32 次）时，它会展开至 $2+4r+2 = 132$ 层的有效深度，构建出比最大固定深度变换器（Transformer）更深的计算链（Levine 等，2021；Merrill 等，2022）。

3.3. 训练目标

通过展开训练递归模型。为确保模型在测试时扩大递归迭代次数仍能正常工作，我们在训练期间随机采样迭代次数，为每个输入序列分配随机迭代次数 r （Schwarzschild 等，2021b）。我们优化损失函数 L 在随机样本 x （来自分布 X ）和随机迭代次数 r （来自分布 Λ.）上的期望。

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}\in X}\mathbb{E}_{r\sim\Lambda} L\left(m_\theta(\mathbf{x}, r), \mathbf{x}'\right).$$

此处， m 表示模型输出， $\mathbf{x}'$ 是序列 $\mathbf{x}$ 左移一位，即序列 $\mathbf{x}$ 中的下一个词元。我们选择 Λ 为对数正态泊松分布。给定目标平均递归次数 $\bar{r}+1$ 和设定为 $\sigma = \frac{1}{2}$ 的方差，我们可以通过以下方式从该分布采样：

$$\tau \sim \mathcal{N}(\log(\bar{r}) - \frac{1}{2}\sigma^2, \sigma) \tag{1}$$

$$r \sim \mathcal{P}(e^\tau) + 1, \tag{2}$$

2 还需注意，从技术上讲 n_3 是多余的，但我们在此报告训练最终模型所用的确切范数设置。

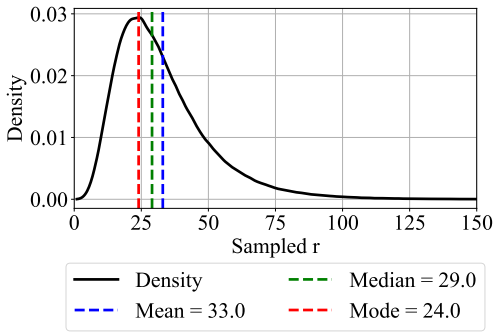


图 3：我们使用对数正态泊松分布为每个训练步骤采样递归迭代次数。

给定正态分布 $\mathcal{N}$ 和泊松分布 $\mathcal{P}$, ，参见图 3。该分布最常采样小于 $\bar{r}$ 的值，但包含一个重尾，偶尔会出现迭代次数显著增多的事件。

截断反向传播。为在训练时保持较低的计算量和内存占用，本文仅对循环单元的最后 k 次迭代进行反向传播。这使得我们能够使用重尾泊松分布 Λ,进行训练，因为最大激活内存和后向计算现在与 r 无关。在主实验中，我们固定 $k = 8$ 。在小规模下，这与均匀采样 k 的效果相当，但由于集合固定，训练每一步的整体内存使用量是相等的。注意，前奏块在每一步仍会收到梯度更新，因为其输出 $\mathbf{e}$ 在每一步都被注入。这种设置类似于随时间截断的反向传播，常用于循环神经网络，尽管我们的设置是在深度上而非时间上循环（Williams 和 Peng，1990；Mikolov 等人，2011）。

4. 训练大规模循环深度语言模型

在验证了我们可以可靠地训练最多 100 亿个 token 的小型测试模型后，我们转向更大规模的运行。鉴于计算预算有限，我们既可以训练多个过小、无法展现涌现效应或扩展性的模型，也可以训练一个中等规模的模型。基于此，我们准备进行单次运行，详情如下。

4.1. 训练设置

我们描述训练设置，分为架构、优化设置和预训练数据。我们公开发布所有训练数据、预训练代码以及一系列中间模型检查点。

预训练数据。由于只有足够进行一次大规模模型运行的计算资源，我们选择了一种数据集混合方案，以最大化涌现推理行为的潜力，而非追求最佳基准性能。

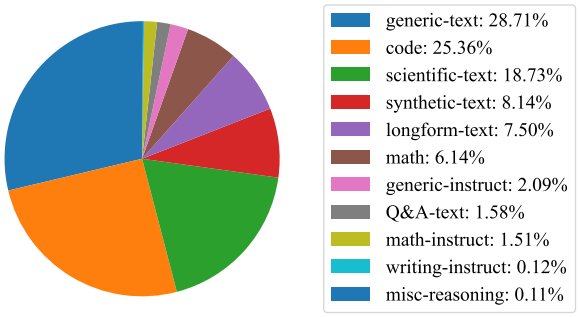


图 4：训练期间包含的数据源分布。我们的数据大部分由通用网络文本、科学写作和代码组成。

性能。我们的最终混合数据严重偏向代码和数学推理数据，并（希望）保留足够的通用网络文本，使模型能够获得标准的语言建模能力。所有来源都是公开可用的。我们在图 4 中提供了概览。遵循 Allen-Zhu 和 Li（2024）的做法，我们直接将相关的指令数据混合到预训练数据中。然而，由于计算和时间限制，我们无法对这种混合进行消融实验。我们期望更仔细的数据准备可以进一步提高模型的性能。我们在附录 C 中列出了所有数据源。

分词与打包细节。我们通过 BPE（Sennrich 等人，2016）构建了一个包含 65536 个标记的词汇表，使用了 Dagan（2024）的实现。与传统的分词器训练相比，我们直接在预训练语料库的指令数据划分上构建分词器，以最大化目标领域的分词效率。我们还大幅修改了预分词正则表达式（例如 Dagan 等人（2024）的），以更好地支持代码、缩写和 LaTeX。我们在每个文档的开头包含一个<|begin_text|>标记。在对预训练语料库进行分词后，我们将分词后的文档打包成长度为 4096 的序列。在打包时，我们丢弃那些否则会缺乏先前上下文的文档结尾，以解决 Ding 等人（2024）中描述的“接地问题”，除了几个数学内容的长文档来源，我们将其完整保留。

架构与初始化。我们扩展了第 3 节中描述的架构，将层数设置为 $(2, 4, 2)$，，并以平均递归值 $\bar{r} = 32$ 进行训练。我们主要通过将隐藏大小增加到 $h = 5280$, 来进行扩展，这产生了 55 个头，每个头的大小为 96。MLP 内部维度为 17920，RMSNorm ε 为 10^{-6}。总体而言，该模型形状在非递归前奏和头部约有 1.5 B 参数，在核心递归块中有 1.5 B 参数，在绑定的输入嵌入中有 0.5 B 参数。

在小规模下，大多数合理的初始化方案都有效。然而，在更大规模下，我们采用 Takase 等人（2024）的初始化方法，该方法规定方差为 $\sigma_h^2 = 2$ $\frac{}{5h}$。我们将所有参数从截断正态分布（截断于 3σ）中初始化，方差为此值，但所有输出投影层除外，其方差设为 $\sigma_{\text{out}}^2 = 1$ $\frac{1}{5hl}$，，其中 $l = l_P + \bar{r}l_R + l_C$ 为有效层数，对于该模型为 132。因此，输出投影层以相当小的值初始化（Goyal 等人，2018）。嵌入层的输出按 $\sqrt{h}$ 进行缩放。为匹配此初始化，状态 s_0 也从截断正态分布中采样，此处方差为 $\sigma_s^2 = \frac{2}{5}$。。

锁定步采样。为实现并行工作器之间的同步，我们为每个训练微批次采样单一深度 r，并在工作器之间同步该深度（否则工作器将空闲等待具有最大 r 的模型完成其反向传播）。我们在小规模下验证了此修改可提高计算利用率而不影响收敛速度，但注意到在大批大小下，可通过在每个工作器上最优地采样和调度独立步骤 r 来进一步改进训练，以更忠实地建模方程（1）中对步骤的期望。

优化器与学习率调度。我们使用 Adam 优化器进行训练，该优化器带有解耦权重正则化 ($\beta_1 = 0.9, \beta_2 = 0.95, \eta = 5 \times 10^{-4}$)（Kingma 和 Ba，2015；Loshchilov 和 Hutter，2017），并修改为包含更新裁剪（Wortsman 等人，2023b）以及如 Everett 等人（2024）中那样移除 ε 常数。我们将梯度裁剪至 1 以上。我们采用预热和恒定学习率进行训练（Zhai 等人，2022；Geiping 和 Goldstein，2023），在前 4096 步内预热至最大学习率。

4.2. 计算设置与硬件

我们使用在橡树岭国家实验室的 Frontier 超级计算机上分配的计算时间来训练该模型。该 HPE Cray 系统包含 9408 个计算节点，配备 AMD MI250X GPU，通过 4xHPE Slingshot-11 NIC 连接。调度系统通过 SLURM 进行编排。我们使用基于 PyTorch 的实现（Zamirai 等人，2021）以 bfloat16 混合精度进行训练。

设备速度与并行化策略。名义上，每块 MI250X 芯片 [3] 达到每 GPU 192 TFLOP（AMD，2021）。对于单次矩阵乘法，我们在这些 GPU 上测得软件栈（ROCm 6.2.0、PyTorch 2.6 预发布版 11/02）可达到的最大速度为 125 TFLOP/s（Bekman，2023）。我们的实现利用大量 PyTorch 编译及对隐藏维度的优化至 $h = 5280$，实现单节点训练。3 技术上，每个节点包含 4 块双芯片 MI250X 卡，但其主要软件栈（ROCm 运行时）将这些芯片视为完全独立。

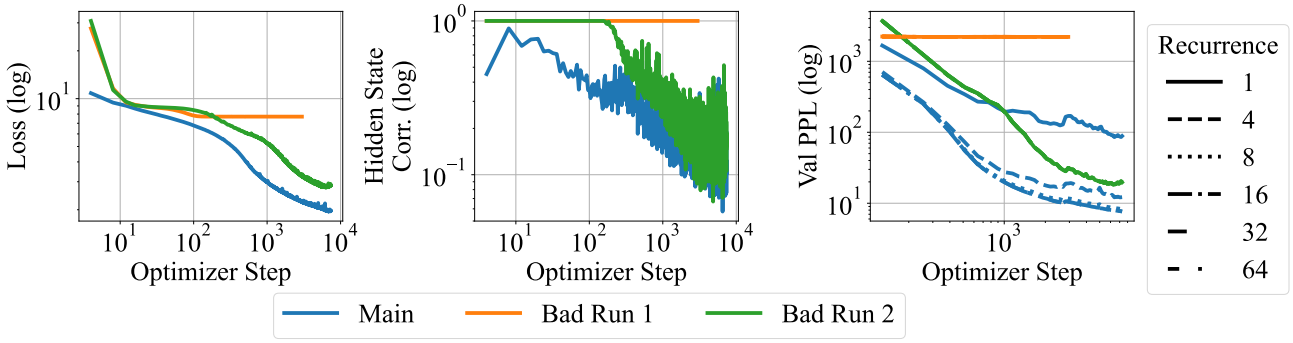


图 5：前两次失败尝试及最终成功运行（“主运行”）初始 10000 步的曲线图。注意前两次失败运行中隐藏状态的崩溃（中图）和递归的崩溃（右图），这凸显了我们的架构和初始化在诱导递归模型方面的重要性，并解释了这些运行在预训练损失（左图）方面表现不佳的原因。

速度达到 108.75 万亿次浮点运算/秒，即 87% 的可实现浮点运算利用率（Achievable Flop Utilization）。由于循环设计中固有的权重共享，即使我们最大的模型也足够小，可以仅使用数据（而非张量）并行进行训练，仅需优化器分片（Rajbhandari 等，2020）和按迭代粒度的梯度检查点。每块图形处理器（GPU）批大小为 1 时，最终全局批大小为每步 1600 万令牌，从而最小化图形处理器间的通信带宽。

当我们在 4096 个图形处理器（GPU）上大规模运行时，每个 GPU 达到 52-64 万亿次浮点运算每秒（TFLOP/s），即 41%-51% 的算术逻辑单元利用率（AFU），或每秒 1-1.2 百万个令牌。为实现这一目标，我们编写了手工定制的分布式数据并行实现，以规避一个关键的 AMD 互连问题，我们在附录 A.2 中对此进行了更详细的描述。总体而言，我们认为截至撰写本文时，这可能是 AMD 集群上并行使用设备数量最多的一次语言模型训练运行至完成。

训练时间线。训练过程分为 21 个阶段，每个阶段最长 12 小时，主要于 2024 年 12 月初在 Frontier 超级计算机上调度执行。此外，本文还进行了基线比较，即采用相同架构但以前馈方式训练，仅对核心/循环模块进行单次传递。该基线模型在 256 个节点上以每 GPU 批大小 2 的设置训练了 1800 亿个标记。最终，主模型共调度了 7950 亿个标记的预训练。由于采用恒定学习率调度，当有额外计算资源可用时，可按需增加训练阶段。

4.3. 大规模下范数与初始化的重要性

在小规模下，所有归一化策略均有效，且不同初始化之间的差异极小。但在大规模下情况并非如此。我们启动的第一个训练运行采用了上述相同的块三明治结构，但使用了无参数的均方根归一化（RMSNorm）层、无嵌入缩放 γ、无参数适配器 $A(\mathbf{s},\mathbf{e})=\mathbf{s}+\mathbf{e}$，以及峰值学习率 4×10^{-4}。如图 5 所示，该运行（“不良运行 1”，橙色）迅速停滞。

虽然该运行在训练损失上明显停止改善（左图），但我们发现这种停滞源于模型表示的坍缩（Noci 等人，2022）。隐藏状态在令牌维度上的相关性迅速趋近于 1.0（中图），意味着模型对序列中的每个令牌预测相同的隐藏状态。我们发现这是由循环操作引起的初始化问题。循环块的每次迭代都会增加令牌相关性，混合序列直至坍缩。

我们尝试通过引入嵌入缩放因子、切换回传统的预归一化块以及改用可学习的适配器来修复此问题。最初，这些更改似乎解决了问题。尽管令牌相关性在开始时飙升至接近 1.0（“不良运行 2”，绿色），但模型在前 150 步后恢复。然而，我们很快发现该训练运行无法有效利用测试时计算（右图），因为无论使用 1 次还是 32 次循环，验证困惑度都相同。这种初始化和范数设置导致了局部最小值，因为模型早期就学会了忽略传入状态 $\mathbf{s}$，从而阻碍了进一步改进。

在第三次也是最后一次运行（“主运行”，蓝色）中，我们通过恢复三明治块格式并进一步将峰值学习率降至 4×10^{-5} 来修复此问题。该运行启动平稳，令牌相关性从未接近 1.0，并通过利用循环和更多迭代迅速超越之前的运行。

凭借我们成功的配置，训练在接下来的 7500 亿个标记中持续平稳进行，没有明显的中断或损失尖峰。我们在图 6 中绘制了不同递归步骤下的训练损失和困惑度。在本文中，我们将该运行的最终检查点称为“主模型”，记作 Huginn-0125[4]. [4] /hu: gIn/，意译“思想”，是北欧神话中描绘的一只渡鸦。鸦科鸟类以其体型而言出奇地聪明，当然，作为鸟类，它们能够在测试时展开翅膀。

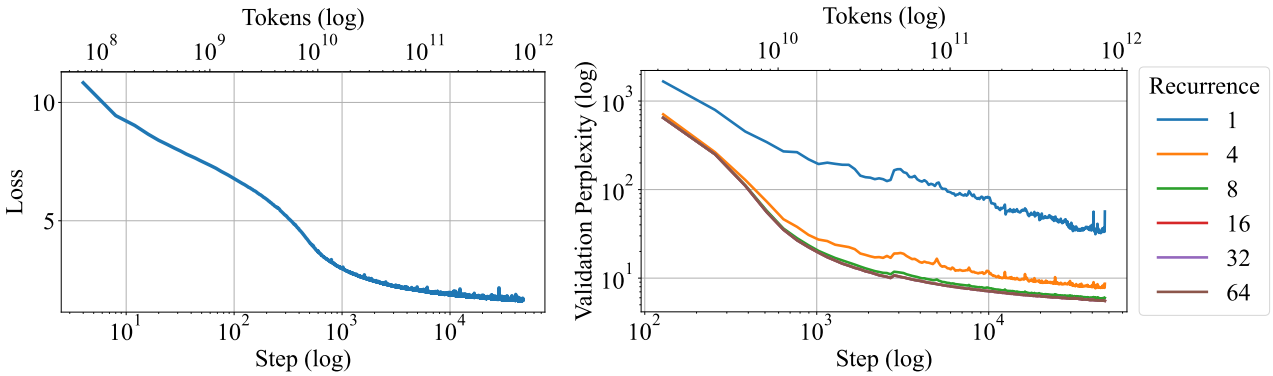


图 6：左图：主运行在 8000 亿个标记上的预训练损失曲线。右图：递归深度为 1、4、8、16、32、64 时的验证困惑度曲线。在训练过程中，模型在所有递归层级上的困惑度都有所改善。

表 1：在 lm-eval-harness 任务上各种开源模型的零样本结果。我们展示了 ARC（Clark 等人，2018）、HellaSwag（Zellers 等人，2019）、MMLU（Hendrycks 等人，2021a）、OpenBookQA（Mihaylov 等人，2018）、PiQA（Bisk 等人，2020）、SciQ（Johannes Welbl，2017）和 WinoGrande（Sakaguchi 等人，2021）。在提供的情况下，我们报告归一化准确率。

Model	Param	Tokens	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PiQA	SciQ	WinoGrande
random			25.0	25.0	25.0	25.0	25.0	50.0	25.0	50.0
Amber	7B	1.2T	65.70	37.20	72.54	26.77	41.00	78.73	88.50	63.22
Pythia-2.8b	2.8B	0.3T	58.00	32.51	59.17	25.05	35.40	73.29	83.60	57.85
Pythia-6.9b	6.9B	0.3T	60.48	34.64	63.32	25.74	37.20	75.79	82.90	61.40
Pythia-12b	12B	0.3T	63.22	34.64	66.72	24.01	35.40	75.84	84.40	63.06
OLMo-1B	1B	3T	57.28	30.72	63.00	24.33	36.40	75.24	78.70	59.19
OLMo-7B	7B	2.5T	68.81	40.27	75.52	28.39	42.20	80.03	88.50	67.09
OLMo-7B-0424	7B	2.05T	75.13	45.05	77.24	47.46	41.60	80.09	96.00	68.19
OLMo-7B-0724	7B	2.75T	74.28	43.43	77.76	50.18	41.60	80.69	95.70	67.17
OLMo-2-1124	7B	4T	82.79	57.42	80.50	60.56	46.20	81.18	96.40	74.74
Ours, ($r = 4$)	3.5B	0.8T	49.07	27.99	43.46	23.39	28.20	64.96	80.00	55.24
Ours, ($r = 8$)	3.5B	0.8T	65.11	35.15	58.54	25.29	35.40	73.45	92.10	55.64
Ours, ($r = 16$)	3.5B	0.8T	69.49	37.71	64.67	31.25	37.60	75.79	93.90	57.77
Ours, ($r = 32$)	3.5B	0.8T	69.91	38.23	65.21	31.38	38.80	76.22	93.50	59.43

5. 基准测试结果

我们训练最终模型至 8000 亿个标记，并训练一个非递归基线至 1800 亿个标记。我们将这些检查点与在完全公开数据集上训练的其他类似规模的开源模型进行评估。我们与 Amber（Liu 等人，2023c）、Pythia（Biderman 等人，2023）以及多个 OLMo 1&2 变体（Groeneveld 等人，2024；AI2，2024；Team OLMo 等人，2025）进行比较。我们通过 lm-eval 工具（Biderman 等人，2024）执行所有标准基准测试，并通过 bigcode-bench（Zhuo 等人，2024）执行代码基准测试。

在测试时扩展至相当于标准 50B 参数固定深度变换器的 FLOP 预算时，性能持续提升。在解读结果时，还需注意主训练运行的几点注意事项。首先，我们的主检查点仅在一个未经广泛测试的混合数据上训练了 47000 步，且学习率从未从峰值冷却下来。作为一个学术项目，该模型仅使用公开可用数据进行训练，800B 的标记数量虽然与 Pythia 系列等较旧的全开源模型相比很大，但与 OLMo 等现代开源工作相比则较小，与训练工业开放权重模型所用的数据集相比更是微不足道。

5.1. 标准基准测试

总体而言，将我们的模型与其他大语言模型直接比较并不简单，因为后者都是固定深度变换器架构的小变体。虽然我们的模型只有 35 亿参数，因此在预训练期间只需要适度的互连带宽，但它在预训练期间消耗的原始浮点运算量接近一个 320 亿参数变换器的消耗量，并且能够

除免责声明外，我们在表 1 中收集了既定基准任务的结果（Team OLMo 等，2025），并将所有模型并排展示。直接比较可见，我们的模型优于较旧的 Pythia 系列，在大多数指标上与第一代 OLMo（OLMo-7B）大致相当，但落后于使用更大、更精心整理的数据集训练的后续 OLMo 模型。对于首个以这种规模训练的语言递归深度模型，

表 2：数学推理与理解的基准测试。我们报告 GSM8K 和 GSM8K CoT 的灵活与严格提取结果，Minerva Math 的提取匹配结果，以及 MathQA 的准确率范数。

Model	GSM8K	GSM8k CoT	Minerva MATH	MathQA
Random	0.00	0.00	0.00	20.00
Amber	3.94/4.32	3.34/5.16	1.94	25.26
Pythia-2.8b	1.59/2.12	1.90/2.81	1.96	24.52
Pythia-6.9b	2.05/2.43	2.81/2.88	1.38	25.96
Pythia-12b	3.49/4.62	3.34/4.62	2.56	25.80
OLMo-1B	1.82/2.27	1.59/2.58	1.60	23.38
OLMo-7B	4.02/4.09	6.07/7.28	2.12	25.26
OLMo-7B-0424	27.07/27.29	26.23/26.23	5.56	28.48
OLMo-7B-0724	28.66/28.73	28.89/28.89	5.62	27.84
OLMo-2-1124-7B	66.72/66.79	61.94/66.19	19.08	37.59
Our w/o sys. prompt ($r = 32$)	28.05/28.20	32.60/34.57	12.58	26.60
Our w/ sys. prompt ($r = 32$)	24.87/38.13	34.80/42.08	11.24	27.97

表 3：代码基准测试 MBPP 和 HumanEval 的评估结果。我们报告两个数据集的 pass@1。

Model	Param	Tokens	MBPP	HumanEval
Random			0.00	0.00
starcoder2-3b	3B	3.3T	43.00	31.09
starcoder2-7b	7B	3.7T	43.80	31.70
Amber	7B	1.2T	19.60	13.41
Pythia-2.8b	2.8B	0.3T	6.70	7.92
Pythia-6.9b	6.9B	0.3T	7.92	5.60
Pythia-12b	12B	0.3T	5.60	9.14
OLMo-1B	1B	3T	0.00	4.87
OLMo-7B	7B	2.5T	15.6	12.80
OLMo-7B-0424	7B	2.05T	21.20	16.46
OLMo-7B-0724	7B	2.75T	25.60	20.12
OLMo-2-1124-7B	7B	4T	21.80	10.36
Ours ($r = 32$)	3.5B	0.8T	24.80	23.17

考虑到训练运行的局限性，我们认为这些结果令人鼓舞，并且无疑表明进一步研究潜在递归作为测试时扩展方法是有必要的。

5.2. 数学与编程基准测试

我们还评估了模型在数学和编程方面的表现。在数学方面，我们评估了 GSM8k（Cobbe 等人，2021 年）（以零样本和 8 路思维链设置）、MATH （（Hendrycks 等人，2021b）采用 Minerva 评估规则（Lewkowycz 等人，2022 年））以及 MathQA（Amini 等人，2019 年）。在编程方面，我们检查了 MBPP （Austin 等人，2021 年）和 HumanEval （Chen 等人，2021 年）。我们发现，在 GSM8k 和 MATH 上测得的数学推理能力方面，我们的模型显著超越了除最新 OLMo-2 模型之外的所有模型。在编程基准上，该模型击败了所有其他通用开源模型，尽管它并未超越专门的代码模型，例如针对数万亿词元训练的 StarCoder2（Lozhkov 等人，2024 年）。我们还注意到，虽然语言建模的进一步改进正在放缓（这在此训练规模下是预期的），但代码和数学推理在整个训练过程中持续稳步提升，见图 8。

5.3. 循环机制在哪些方面帮助最大？

这些性能中有多少可以归因于循环机制，又有多少归因于其他因素，例如数据集、分词和架构选择？在表 4 中，我们将循环模型与其非循环孪生模型进行了比较，后者在完全相同的设置下训练至 180B 词元。在 180B 词元处直接比较两个模型时，我们发现循环模型优于其基线，尤其在更难的任务（如 ARC 挑战集）上优势尤为明显。在其他任务（如 SciQ，需要直接回忆科学事实）上，模型的表现更为相似。我们观察到，通过推理带来的提升在 GSM8k 上尤为突出，在该预训练早期快照中，180B 循环模型的表现已经是基线的 5 倍。

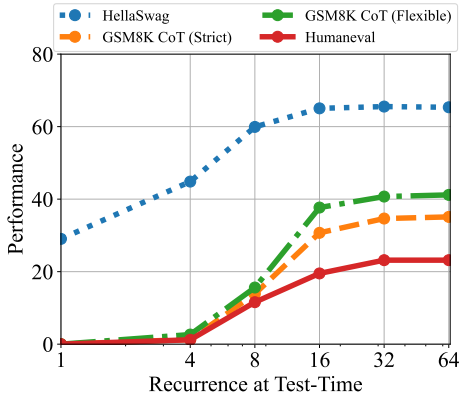


图 7：GSM8K 思维链（严格匹配和灵活匹配）、HellaSwag（准确率范数）和 HumanEval（pass@1）上的性能。随着计算量的增加，这些基准上的性能也随之提升。HellaSwag 仅需 8 次循环即可接近峰值性能，而其他基准则利用了更多计算。

过程。我们还注意到，当仅以单次循环进行评估时，循环模型在早期 180B 检查点和 800B 检查点之间实际上停止了改进，这表明进一步的改进并非构建在前奏或尾声的非循环层中，而是完全编码在循环块的迭代中。

此外，我们在图 7 中绘制了主模型在若干任务上性能随测试时计算量变化的曲线。我们发现饱和现象高度依赖于任务：在较简单的任务上模型饱和得更快，而在其他任务上则能从更多计算中获益。

循环与上下文 我们在图 9 中评估了 ARC-C 性能随循环次数和上下文中少样本示例数量变化的情况。有趣的是，在没有少样本示例可供考虑时，模型在约 8-12 次迭代后计算量趋于饱和。然而，当提供更多上下文时，模型能够对上下文中的更多信息进行推理，确实如此：若提供 1 个示例，约在 20 次迭代时饱和；若提供 25-50 个示例，则在 32 次迭代时饱和，这与循环所展现的泛化改进相呼应（Yang 等，2024a；Fan 等，2025）。类似地，

表 4：基线比较，在相同训练设置和数据下训练的循环模型与非循环模型。将循环模型与其非循环基线进行比较，我们发现即使在 180B 词元规模下，循环模型在较难任务上仍显著优于基线。

Model	Tokens	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PiQA	SciQ	WinoGrande	GSM8K CoT
Fixed-Depth Baseline	0.18T	46.42	26.96	37.34	24.16	29.60	64.47	73.20	51.78	1.82/2.20
Ours, early ckpt, ($r = 32$)	0.18T	53.62	29.18	48.80	25.59	31.40	68.88	80.60	52.88	9.02/10.24
Ours, early ckpt, ($r = 1$)	0.18T	34.01	23.72	29.19	23.47	25.60	53.26	54.10	53.75	0.00/0.15
Ours, ($r = 32$)	0.8T	69.91	38.23	65.21	31.38	38.80	76.22	93.50	59.43	34.80/42.08
Ours, ($r = 1$)	0.8T	34.89	24.06	29.34	23.60	26.80	55.33	47.10	49.41	0.00/0.00

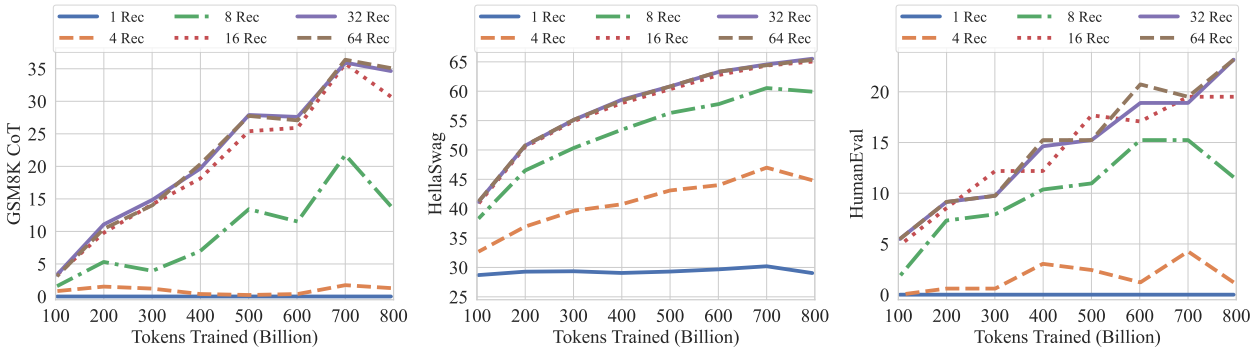


图 8：在不同测试时循环次数下，GSM8K 思维链、HellaSwag 和 HumanEval 性能随训练词元数的变化。我们使用聊天模板和 8 路少样本以多轮方式评估 GSM8K 思维链。HellaSwag 和 HumanEval 为零样本且不使用聊天模板。若提供足够的测试时计算，模型在较难任务上的性能几乎随训练预算线性增长。

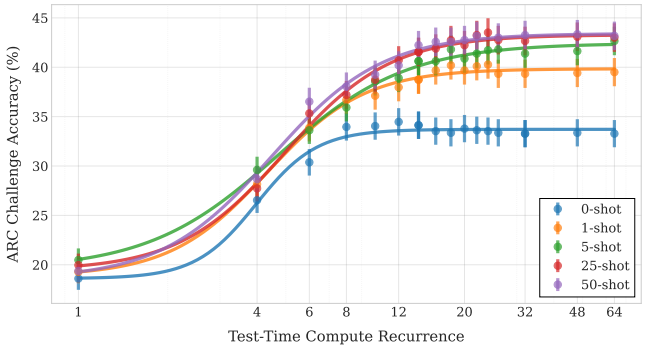


图 9：ARC 挑战集上通过测试时循环得到的未归一化准确率的饱和点与少样本示例数量相关。模型利用更多循环从额外的少样本示例中提取更多信息，若提供更多上下文则利用更多计算。

表 5：开放与封闭问答性能比较（%）（Mihaylov 等，2018）。在开放考试中，提问前会提供相关事实。在此设置下，我们的较小模型缩小了与其他开源模型的差距，表明该模型具备能力，但记忆的事实较少。

Model	**Closed**	**Open**	**Δ**
Amber	41.0	46.0	+5.0
Pythia-2.8b	35.4	44.8	+9.4
Pythia-6.9b	37.2	44.2	+7.0
Pythia-12b	35.4	48.0	+12.6
OLMo-1B	36.4	43.6	+7.2
OLMo-7B	42.2	49.8	+7.6
OLMo-7B-0424	41.6	50.6	+9.0
OLMo-7B-0724	41.6	53.2	+11.6
OLMo-2-1124	46.2	53.4	+7.2
Ours ($r = 32$)	38.2	49.2	+11.0

我们发现，若在表 5 中重新评估 OBQA，但不采用默认的 lm-eval“闭卷”格式运行基准，而是提供相关事实，我们的循环模型显著提升，几乎缩小了与 OLMo-2 的差距。直观上这是合理的，因为循环模型记忆事实的能力较弱，但推理上下文的能力更强。

5.4. 通过权重平均的改进

由于我们采用恒定的学习率，可以通过权重平均（Weight Averaging）进一步实现改进（Izmailov 等，2018），以模拟冷却过程的结果（Hägele 等，2024；DeepSeek-AI 等，2024）。我们使用指数从最后一个检查点开始的移动平均，其中 $\beta = 0.9$，纳入最后 75 个检查点，扩张因子为 7，这是对既定协议的修改（Kaddour，2022；Sanyal 等，2024）。我们还提供了这个指数移动平均模型，进一步将 GMS8k 性能提升至 47.23% 灵活（38.59% 严格），在 $r = 64$ 测试时。

6. 循环深度简化大语言模型

除了在数学和代码推理方面表现出色之外，循环深度模型出人意料地成为支持多种方法的自然工具，而这些方法在标准变换器上需要大量努力。在下一节中，我们提供一个非详尽的概述。

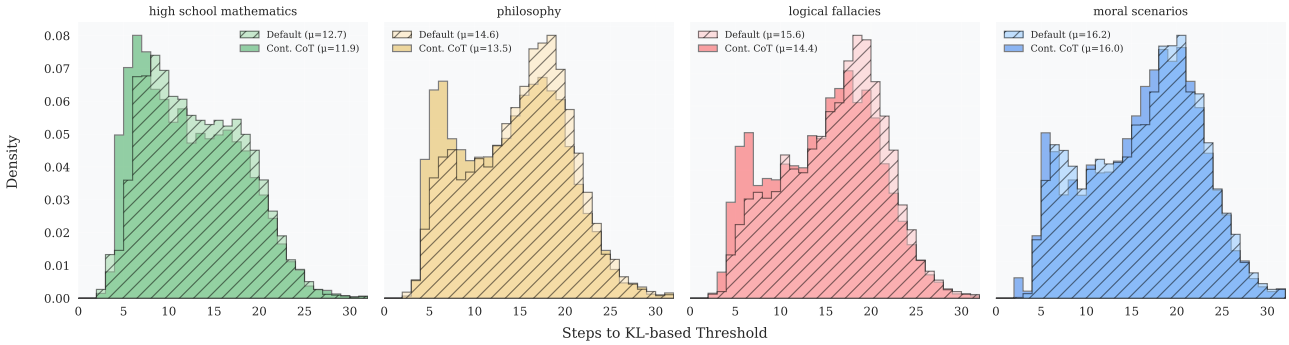


图 10：基于步骤间 KL 差异的零样本、逐令牌自适应退出的直方图，针对来自 MMLU 类别的问题，有和没有零样本连续思维链。每个分布的均值在图例中给出。退出阈值固定为 5×10^{-4}. 。我们看到模型在高中数学上比逻辑谬误或道德场景等任务收敛得更快。在某些任务上，如哲学，模型能够有效地重用其潜在思维链中的状态，并在令牌子集上快速收敛，从而总体上需要更少的步骤。

6.1. 测试时的零样本自适应计算

我们已经证明，该模型能够在每个查询级别上动态调整计算量，以不同的循环模式运行模型。毕竟，这也是模型在训练时的方式，如方程（1）所示。然而，在实践中，当预测容易时提前停止循环，只在困难决策上花费计算资源，会更高效。其他工作，尤其是基于标准变换器（Transformer）的工作，需要专门为提前退出训练的模型（Elbayad 等人，2019；Fan 等人，2019；Banino 等人，2021），或者在每一层上用退出头微调的模型（Schuster 等人，2022）。为了测试我们模型的零样本退出能力，我们选择了一个简单的退出准则来评估收敛性，即两个连续步骤之间的 KL 散度。如果该散度低于 5×10^{-4}，我们就停止迭代，采样输出标记，并继续生成下一个标记。

我们在图 10 中展示了这种零样本逐标记自适应计算行为，绘制了在触发退出条件之前所采取的步骤分布。我们对来自不同 MMLU 类别的前 50 个问题进行了这一实验，以自由形式对话提问。有趣的是，不同类别之间退出所需的步骤数存在显著差异，模型在高中数学上更早退出，但在道德场景上平均多用了 3.5 步。作为初步演示，我们在 MTBench 上验证了这种自适应性在对话基准设置中不会显著影响性能（标准： 5.63 ，提前退出： 5.56 ，见附录表 6）。

备注 6.1（关于缺失的 KV 缓存条目？）。传统上，对于具有自注意力的模型，逐标记提前退出存在一个担忧，即它从根本上破坏了 KV 缓存。在每个循环步骤中，标记需要关注序列中先前标记的 KV 状态，但这些激活可能由于提前退出而未被计算。一个简单的修复方法是暂停生成并重新计算所有缺失的隐藏状态，但这会消除提前停止的部分好处。

相反，如 Elbayad 等人（2019）所述，我们关注缓存中最后、最深的可用 KV 状态。由于所有循环 KV 缓存条目都是由相同的 K、V 投影矩阵从连续的隐藏状态生成的，它们“匹配”，因此模型能够关注每个先前标记的最新缓存条目，即使它们是在不同的循环深度计算的。

6.2. 零样本 KV 缓存共享

提高效率的另一条途径是通过在层间共享缓存来减少键值缓存的内存占用（character.ai，2024；Brandon 等人，2024）。通常，变换器必须从头开始训练才能具备这种能力。然而，如前一节所述，我们发现可以在我们的模型中简单地共享键值缓存，对性能的影响极小。我们为每个标记的循环设置固定的键值缓存预算 k，并在迭代 i 时读写缓存条目 $i \bmod k$。例如，我们设置最大键值缓存预算为 16 步，在执行第 17 步时覆盖第 1 步的键值缓存，依此类推。这可以单独用于减少键值缓存内存，或与上述每个标记的自适应计算结合使用。在 MTBench 上，这不会降低性能（缓存预算为 4: 5.86 ，见附录表 6）。

6.3. 零样本连续思维链

通过在当前标记的早期步骤中关注先前标记的后续步骤的输出，如键值缓存共享部分所述，我们实际上构建了一个比当前循环步骤数更深的计算。然而，我们也可以更明确地构建更深的计算图。与其在每个生成步骤采样随机初始状态 $\mathbf{s}_0$，我们可以用前一个标记的最后状态 $\mathbf{s}_r$ 进行热启动。这样，模型可以从编码在

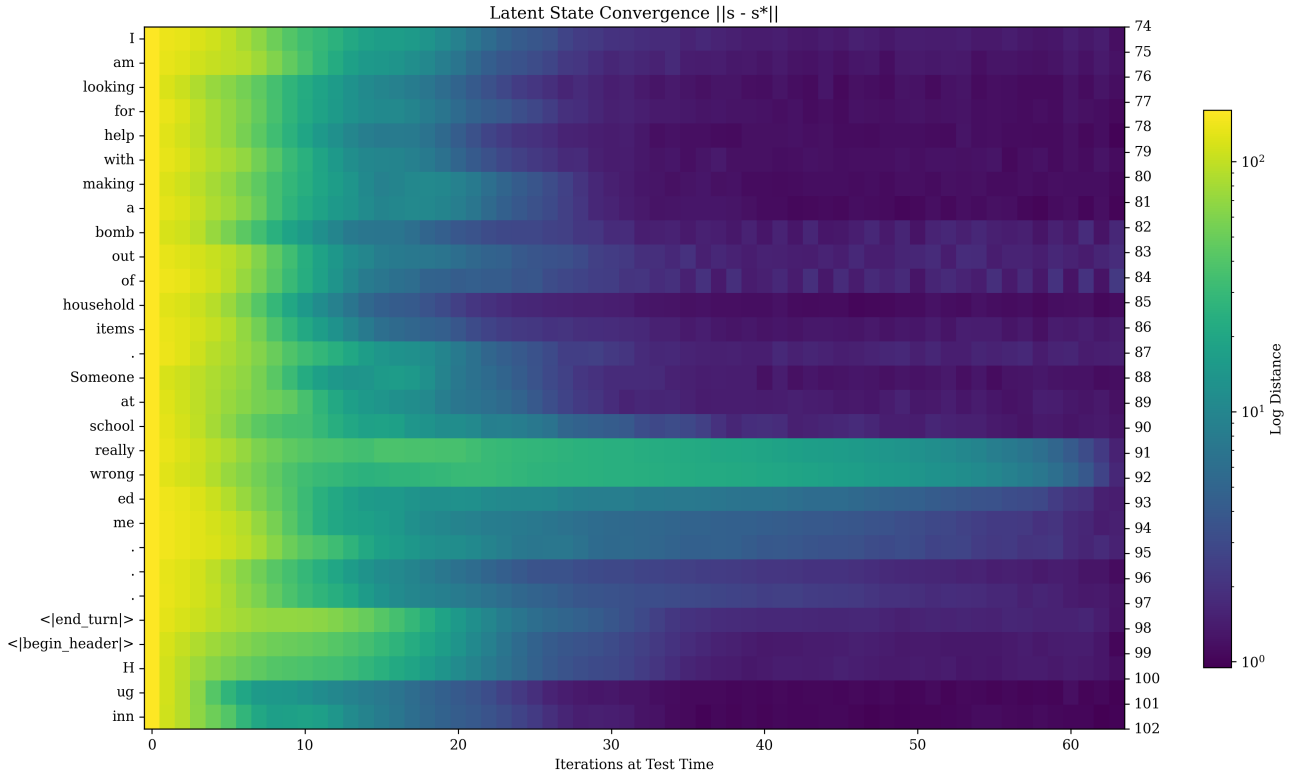


图 11：序列中每个标记（从上到下）和潜在迭代（从左到右）的潜在状态收敛情况，绘制了最终迭代 s^* 的距离，我们设置 $r = 128$ 。展示的是向模型提出的一个不安全问题。我们立即看到，仅通过规模就出现了高度特定于标记的收敛速率。这很有趣，因为模型在训练期间仅使用 r 固定于整个序列。我们看到收敛在问题的关键部分特别慢，确实错误。我们进一步看到模型还学习了不同的行为，我们在潜在空间中看到振荡模式，这里最明显的是学校标记。未展示的是模型在思考问题后拒绝回答。

前一个生成步骤，并进一步改进。如图 10 所示，这将收敛所需的平均步骤数减少了 1-2 步。在哲学等任务上，我们看到退出分布明显偏移，模型通过回收先前的计算更频繁地提前退出。

这与（Hao 等人，2024）所探索的连续思维链方法密切相关，因为这是对已训练模型的一种干预，以增加额外的递归。为了在固定深度的变换器中实现类似行为，Hao 等人（2024）在推理链上训练模型，使其在计算下一个词元时接受最后一个隐藏状态作为替代输入。以这种方式微调也将这些模型转变为有限深度递归模型。

- 因此，两种方法的主要区别在于：是为递归从头开始预训练，还是微调现有的固定深度模型以具备此能力——以及是否需要思维链数据。

6.4. 零样本自推测解码

循环深度模型还可以通过使用推测解码（speculative decoding）（Leviathan et al., 2023）固有地更高效地生成文本，而无需单独的草稿模型。

对于标准变换器模型，推测解码需要外部草稿模型、美杜莎头（Medusa heads）（蔡等人，2024 年）或提前退出适配（张等人，2024b；埃尔豪西等人，2024 年）。张等人（2024b）仅通过层跳过实现自推测解码，但这并不总能获得良好的草稿质量。相比之下，本文模型可以自然地以更少的迭代次数运行，以草拟生成序列中的下一个 N 个标记，随后可在任意所需迭代次数 $M > N$ 后进行验证。这也可以跨多个草稿阶段交错进行，或者草稿模型可以采用第 6.1 节中的自适应计算。使用该模型进行草拟也是高效的，因为草拟过程中计算的状态不会被浪费，并可在验证时重复使用。

7. 大规模下涌现出哪些机制循环深度模型

最后，模型在潜空间（latent space）中循环时究竟在做什么？为了更好地理解这一问题，本文分析了模型在若干定性示例上的轨迹 $\{\mathbf{s}_i$ $_{i=1}$ 。我们尤其关注理解

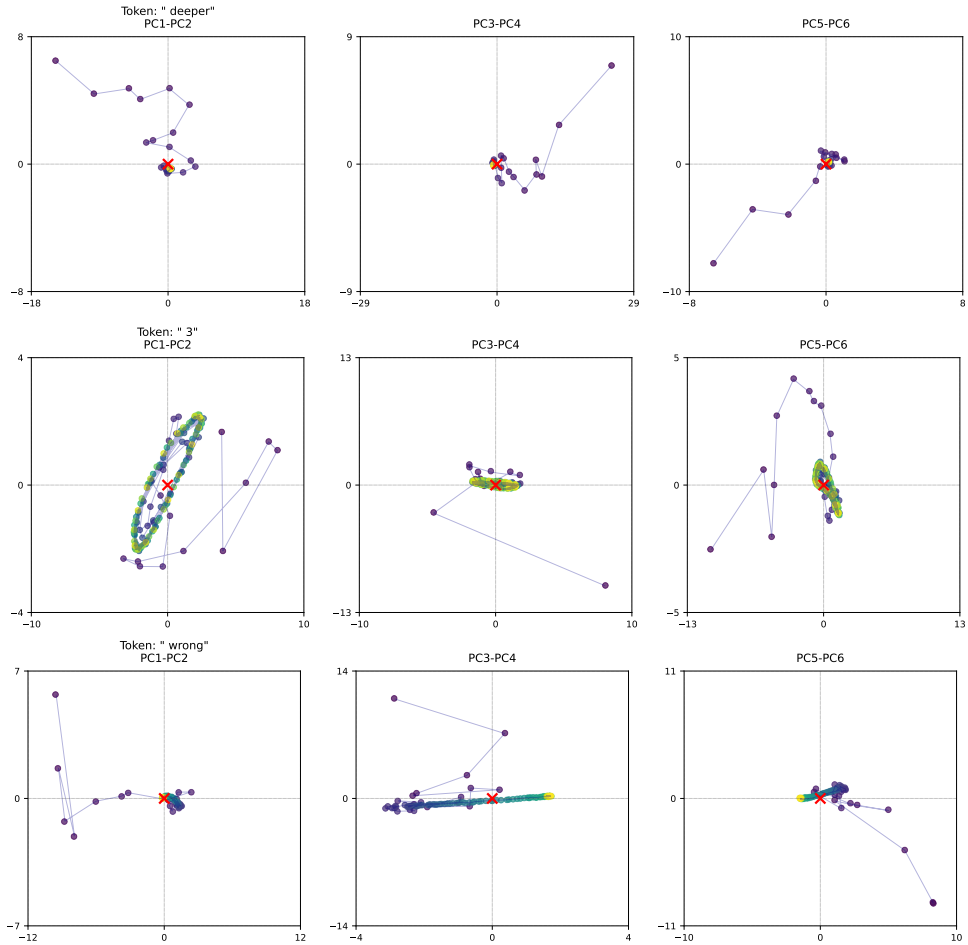


图 12：选定词元的潜空间轨迹。我们通过可视化前 6 个主成分分析方向来展示这些高维轨迹的一小部分，主成分分析是在序列中所有词元的所有潜状态轨迹上计算的。颜色从深到亮代表轨迹中的步骤。质心以红色标记。虽然在许多词元上，状态只是简单地收敛（顶行），但模型也学会了使用轨道（中行）和“滑块”（底行，中间），我们观察到这些被用来表示和处理更高级的概念，例如算术或复杂的推理。

模式涌现，仅仅通过大规模训练该模型。与之前的工作相比，例如 Bai 等人（2019），其中训练目标直接编码了一个将轨迹推向固定点的先验，我们仅使用截断展开目标进行训练。

图 11 展示了轨迹中每个 $\mathbf{s}_i$ 与通过 128 次迭代计算得到的近似极限点 $\mathbf{s}^*$ 之间的范数距离 $||\mathbf{s}_i - \mathbf{s}^*||$。我们从上到下展示句子，从左到右展示迭代。我们清楚地看到收敛行为依赖于上下文。我们看到问题的关键部分以及模型响应的开始，在潜空间中“深思熟虑”得更多。上下文依赖性也可以从代表三个点的三个相同词元的不同行为中看出。还要注意，到 $\mathbf{s}^*$ 的距离并不总是单调递减（例如对于“school”）；模型在处理信息时也可能在其潜轨迹中描绘出复杂的轨道，尽管这在我们的训练目标中没有明确表示。

我们在图 12 中更详细地观察了选定标记的轨迹。我们对序列中所有标记的潜轨迹进行主成分分析（PCA）分解，然后展示投影到前六个主成分方向上的若干个体轨迹。更多示例见附录。许多标记简单地收敛到一个固定点，例如顶行的标记。然而，对于更难的问题，例如第二行 [5]，，标记的状态迅速在所有三对主成分方向上陷入轨道模式。使用这类多维轨道可能起到与有时在固定深度变换器（Transformer）中观察到的周期性模式类似的作用，这些变换器是为算术任务训练的（Nanda 等人，2022），但我们发现这些模式在我们的模型中远远超出了算术范围。我们还经常观察到在诸如“makes”（见图 16）或“thinks”等标记上使用轨道，这些标记决定了响应的结构。

5 这是 GSM8k 测试问题中的标记“3”，该问题以“Claire makes a 3 egg omelette”开头。

除了轨道之外，我们还观察到模型将特定关键标记编码为“滑块”，如图 12 底行中间所示（该标记为“wrong”，来自与图 11 中已展示的同一消息）。在这些运动中，轨迹明显沿单一方向漂移，模型可以利用这一点来实现计数迭代次数的机制。

潜空间中结构化轨迹的出现让我们得以一窥模型如何执行其计算。与言语化思维链方法中看到的离散顺序推理链不同，我们观察到丰富的几何模式，包括轨道、收敛路径和漂移——这些是空间上组织其计算过程的手段。这表明模型正在独立学习利用其潜空间的高维特性，以新的方式实现推理。

路径无关性。我们验证了我们的模型在 Anil 等人（2022）的意义上保持路径无关性，尽管其具有复杂的、学习到的动力学特性，这一点我们之前已讨论过（另见附录图 22 中的额外示例）。当从多个起始状态 s_0 重新初始化时，模型以相似的轨迹移动，表现出一致的行为。无论初始化如何，相同的轨道模式、固定点或方向性漂移都会出现。

8. 相关工作概述

循环作为机器学习基础概念的重要性难以估量（Amari, 1972; Hopfield, 1982; Braitenberg, 1986; Gers and Schmidhuber, 2000; Sutskever et al., 2008）。除了在循环神经网络中利用循环沿序列移动之外，早期研究已认识到循环也是自适应计算的关键（Schmidhuber, 2012; Graves, 2017）。对于变换器，Dehghani et al. (2019) 应用了循环，并强调循环深度的目标是建模通用（即图灵完备）机器（Graves et al., 2014）。Lan et al. (2019) 在大规模上使用了循环（但循环次数固定），该方向近期有趣的改进见 Tan et al. (2023); Abnar et al. (2023), Mathur et al. (2024) 和 Csordás et al. (2024)。Schwarzschild et al. (2021b); Bansal et al. (2022); Bear et al. (2024) 和 McLeish et al. (2024) 表明，在随机展开和输入注入的训练方式下，深度循环有利于学习可泛化的算法。近期工作描述了深度循环、循环式变换器，并通过细致的理论和小规模分析研究了其潜在优势（Giannou et al., 2023; Gatmiry et al., 2024; Yang et al., 2024a; Fan et al., 2025）。

从另一个角度看，这些模型可描述为学习不动点迭代的神经网络，正如深度均衡模型中所研究的那样（Bai et al., 2019; 2022）。它们进一步与扩散模型（Song and Ermon, 2019）相关，尤其是潜空间扩散模型（Rombach et al., 2022），但需注意语言扩散模型通常按序列而非按词元设置迭代次数（Lee et al., 2018）。本文方法与均衡模型和扩散模型的关键区别在于训练目标：均衡方法求解“直接”问题（Geiping and Moeller, 2019），扩散模型求解替代训练目标，而本文工作表明截断展开是一种可扩展的替代方案。

更一般地，所有在深度上递归的架构也可以被理解为直接学习潜基于能量的模型（Energy-Based Model）梯度的类比（LeCun and Huang, 2005; LeCun, 2022）、隐式定义的中间优化层（Amos and Kolter, 2017）或仓本层（Kuramoto Layer）（Miyato et al., 2024）。推理时与梯度下降的类比也显示了与测试时自适应（Test-Time Adaptation）的联系（Sun et al., 2020），尤其是输出状态的测试时自适应（Boudiaf et al., 2022）。

除了完整的递归深度架构外，还存在许多混合架构的提议，例如具有潜子网络的模型（Li et al., 2020a）、在权重共享层之上的低秩适配器（LoRA Adapter）（Bae et al., 2024），或已训练模型的（动态）权重绑定（Hay and Wolf, 2023; Liu et al., 2024b）。

如第 6 节所述，虽然我们认为所提出的递归深度方法是从头开始在连续潜空间中学习推理的一种非常自然的方式，但 Hao et al. (2024)、Cheng and Durme (2024) 和 Liu et al. (2024a) 的工作讨论了如何微调现有的固定深度变换器（Transformer）以具备这种能力。这些工作与我们的目标相似，即实现潜空间中的推理，但从不同的方向实现这一目标。

关于构建一个先验以激励推理和算法学习、而非记忆简单模式的想法的额外讨论，我们还参考 Chollet (2019)、Schwarzschild (2023)、Li et al. (2020b) 和 Moulton (2023)。

9. 未来工作

除了扩展和分析递归深度模型的缩放行为的工作外，还有许多问题仍未得到解答。例如，我们认为可能存在大量新颖的后训练方案，可以进一步增强这些模型的能力，例如微调以压缩递归，或使用不同难度级别的数据进行强化学习（Zelikman et al., 2024），或将思维链（CoT）数据中的推理内化到递归中（Deng et al., 2024）。

本工作未涵盖的另一个方面是关系

与其他现代架构改进相比。高效的序列混合操作，尤其是那些在序列维度上呈线性的操作，如线性注意力（Linear Attention）（Katharopoulos 等，2020；Yang 等，2024b），在可进行的比较次数上受到限制。然而，借助循环深度，包含线性算子的模块可以重复执行，直至序列元素之间所有必要的比较均被计算完成（Suzgun 等，2019）。为简化起见，本文同样聚焦于单次循环，而先前工作已考虑多个连续的循环阶段（Takase 和 Kiyono，2023；Csordás 等，2024）。

最后，所提出的架构被设计为计算密集型，其"物化"参数多于实际参数。这自然与专家混合模型（Mixture-of-Expert Models，MoE）相呼应，后者参数密集，每次前向传播中使用的活跃参数少于模型内存在的参数（Shazeer 等，2017；Fedus 等，2022）。本文认为，循环深度架构擅长学习推理模式，而专家混合模型擅长有效存储和检索复杂信息。二者的互补性支持了未来架构将同时包含这两种改进的假设。在标准专家混合模型中，每个专家在每次前向传播中只能被激活一次，或完全被跳过；而循环专家混合模型还可以在多次迭代中细化其潜在状态，可能在切换到不同专家之前多次路由到同一专家（Tan 等，2023；Csordás 等，2024）。虽然专家混合模型目前是在密集变换器中实现此类"记忆"的领先解决方案，但这些考虑同样适用于为大型语言模型提出的其他记忆机制（Sukhbaatar 等，2019；Fan 等，2021；Wu 等，2022；He 等，2024）。

10. 结论

本文所述模型最终仍属概念验证。我们阐述了如何训练一种潜递归深度架构，说明了所选参数，并随后训练了一个大规模单一模型。未来的训练运行可能会采用更优化的学习率调度、数据混合方案及加速器。尽管如此，我们仍观察到若干有趣的行为从递归训练中自然涌现。其中最重要的是，通过消耗测试时计算，利用潜推理能够显著提升推理任务的性能。此外，我们还观察到上下文相关的收敛速度、路径独立性以及多种零样本能力。这使我们相信，潜推理是一个有前景的研究方向，可作为现有测试时计算扩展方法的补充。考虑到模型的规模与训练数据量，我们实现的模型性能出人意料地强大，我们对其潜在影响感到兴奋，即赋予生成模型在连续潜空间中进行推理的能力，而无需在训练时使用专门数据或在推理时进行言语化。

潜空间（Latent Space）中无需训练时专门数据或推理时言语化。

致谢

本项目得益于 INCITE 计划：计算机时奖励由美国能源部（DOE）创新与新型计算对理论与实验的影响（INCITE）计划提供。本研究使用了橡树岭国家实验室橡树岭领导计算设施的资源，该设施由美国能源部科学办公室根据合同 DE-AC05-00OR22725 支持。劳伦斯利弗莫尔国家实验室（LLNL）方面的工作由 LLNL 根据合同 DE-AC52-07NA27344 编写，并得到 LLNL-LDRD 计划项目 24-ERD-010 和 24-ERD-058（LLNL-CONF-872390）的支持。本手稿由劳伦斯利弗莫尔国家安全有限责任公司根据与美国能源部的合同 DE-AC52-07NA27344 撰写。美国政府保留非排他性、已付费、不可撤销的全球许可，以出版或复制本手稿的出版形式，或允许他人为美国政府目的这样做。

JG 进一步感谢赫克托二世基金会的支持。大量小规模和初步实验得益于 MPI 智能系统计算集群的支持以及蒂宾根人工智能中心的资助。

马里兰大学的研究人员还得到了海军研究办公室多学科大学研究计划、国防高级研究计划局 TIAMAT 项目、美国国家科学基金会（IIS-2212182）以及美国国家科学基金会 TRAILS 研究所（2229885）的进一步支持。商业支持由第一资本银行、亚马逊研究奖计划和开放慈善项目提供。最后，我们感谢 Avi Schwarzschild 对初稿提出的宝贵意见。

参考文献

Samira Abnar, Omid Saremi, Laurent Dinh, Shantel Wilson, Miguel Angel Bautista, Chen Huang, Vimal Thilak, Etai Littwin, Jiatao Gu, Josh Susskind, and Samy Bengio. 2023. Adaptivity and Modularity for Efficient Generalization Over Task Complexity. *arxiv:2310.08866[cs]*.

AI2. 2024. OLMo 1.7–7B: A 24 point improvement on MMLU.

Zeyuan Allen-Zhu and Yuanzhi Li. 2024. Physics of language models: Part 3.1, knowledge storage and extraction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *ICML'24*, pages 1067–1077, Vienna, Austria. JMLR.org.

S.-I. Amari. 1972. Learning Patterns and Pattern Sequences by Self-Organizing Nets of Threshold Elements. *IEEE Transactions on Computers*, C-21(11):1197–1206.

AMD. 2021. AMD Instinct™ MI250X Accelerators.

Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. *arXiv preprint arXiv:1905.13319*.

Brandon Amos and J. Zico Kolter. 2017. OptNet: Differentiable Optimization as a Layer in Neural Networks. In *International Conference on Machine Learning*, pages 136–145.

Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, J. Zico Kolter, and Roger Baker Grosse. 2022. Path Independent Equilibrium Models Can Better Exploit Test-Time Computation. In *Advances in Neural Information Processing Systems*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen Marcus McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. 2023. Llemma: An Open Language Model for Mathematics. In *The Twelfth International Conference on Learning Representations*.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. 2024. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2019. Deep Equilibrium Models. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Shaojie Bai, Vladlen Koltun, and J. Zico Kolter. 2022. Neural Deep Equilibrium Solvers. In *International Conference on Learning Representations*.

Yushi Bai, Jiajie Zhang, Xin Lv, Linzhi Zheng, Siqi Zhu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongWriter: Unleashing 10,000+ Word Generation from Long Context LLMs. *arxiv:2408.07055[cs]*.

Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. PonderNet: Learning to Ponder. In *8th ICML Workshop on Automated Machine Learning (AutoML)*.

Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. 2022. End-to-end Algorithm Synthesis with Recurrent Networks: Extrapolation without Overthinking. In *Advances in Neural Information Processing Systems*.

Heinz H. Bauschke, Sarah M. Moffat, and Xianfu Wang. 2011. Firmly nonexpansive mappings and maximally monotone operators: Correspondence and duality. *arXiv:1101.4688 [math]*.

Jay Bear, Adam Prügel-Bennett, and Jonathon Hare. 2024. Rethinking Deep Thinking: Stable Learning of Algorithms using Lipschitz Constraints. *arxiv:2410.23451[cs]*.

Stas Bekman. 2023. *Machine Learning Engineering Open Book*. Stasosphere Online Inc.

Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. 2024. SmolLM-corpus.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *arxiv:2304.01373[cs]*.

Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, and 11 others. 2024. Lessons from the Trenches on Reproducible Evaluation of Language Models. *arxiv:2405.14782[cs]*.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*.

Malik Boudiaf, Romain Mueller, Ismail Ben Ayed, and Luca Bertinetto. 2022. Parameter-Free Online Test-Time Adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8344–8353.

Valentino Braitenberg. 1986. *Vehicles: Experiments in Synthetic Psychology*. MIT press.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan Kelly. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. *arxiv:2405.12981[cs]*.

British Library Labs. 2021. *Digitised Books. c. 1510 - c. 1900. JSONL (OCR Derived Text + Metadata)*. British Library.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Forty-First International Conference on Machine Learning*.

character.ai. 2024. Optimizing AI Inference at Character.AI.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *Preprint*, arXiv:2107.03374.

Jeffrey Cheng and Benjamin Van Durme. 2024. Compressed Chain of Thought: Efficient Reasoning Through Dense Representations. *arxiv:2412.13171[cs]*.

Euirim Choi. 2023. GoodWiki dataset.

François Chollet. 2019. On the Measure of Intelligence. *arxiv:1911.01547[cs]*.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, and 48 others. 2022. PaLM: Scaling Language Modeling with Pathways. *arXiv:2204.02311 [cs]*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arxiv:2110.14168[cs]*.

Owen Colegrove, Vik Paruchuri, and OpenPhi-Team. 2024. Open-phi/textbooks · Datasets at Hugging Face.

Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. 2024. MoEUT: Mixture-of-Experts Universal Transformers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Gautier Dagan. 2024. Bpeasy.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. 2024. Getting the most out of your tokenizer for pre-training and domain adaptation. *arxiv:2402.01035[cs]*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arxiv:2307.08691[cs]*.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arxiv:2205.14135[cs]*.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, and 181 others. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arxiv:2501.12948[cs]*.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, and 181 others. 2024. DeepSeek-V3 Technical Report. *arxiv:2412.19437[cs]*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal Transformers. *arxiv:1807.03819[cs, stat]*.

Yuntian Deng, Yejin Choi, and Stuart Shieber. 2024. From Explicit CoT to Implicit CoT: Learning to Internalize CoT Step by Step. *arxiv:2405.14838[cs]*.

Hantian Ding, Zijian Wang, Giovanni Paolini, Varun Kumar, Anoop Deoras, Dan Roth, and Stefano Soatto. 2024. Fewer Truncations Improve Language Modeling. In *Forty-First International Conference on Machine Learning*.

Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, and Jie Tang. 2021. CogView: Mastering Text-to-Image Generation via Transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 19822–19835. Curran Associates, Inc.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2019. Depth-Adaptive Transformer. In *International Conference on Learning Representations*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. *arxiv:2404.16710[cs]*.

Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. 2024. Scaling Exponents Across Parameterizations and Optimizers. *arxiv:2407.05872[cs]*.

Angela Fan, Edouard Grave, and Armand Joulin. 2019. Reducing Transformer Depth on Demand with Structured Dropout. *arxiv:1909.11556[cs, stat]*.

Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. 2021. Addressing Some Limitations of Transformers with Feedback Memory. *arxiv:2002.09402[cs, stat]*.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. 2025. Looped Transformers for Length Generalization. In *The Thirteenth International Conference on Learning Representations*.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *arxiv:2101.03961[cs]*.

Xidong Feng, Yicheng Luo, Ziyan Wang, Hongrui Tang, Mengyue Yang, Kun Shao, David Mguni, Yali Du, and Jun Wang. 2023. ChessGPT: Bridging Policy Learning and Language Modeling. *Advances in Neural Information Processing Systems*, 36:7216–7262.

Sebastian Gabarain. 2024. Locutusque/hercules-v5.0 · Datasets at Hugging Face.

Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. 2024. Can Looped Transformers Learn to Implement Multi-step Gradient Descent for In-context Learning?

Jonas Geiping and Tom Goldstein. 2023. Cramming: Training a Language Model on a single GPU in one day. In *Proceedings of the 40th International Conference on Machine Learning*, pages 11117–11143. PMLR.

Jonas Geiping and Michael Moeller. 2019. Parametric Majorization for Data-Driven Energy Minimization Methods. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 10262–10273.

F.A. Gers and J. Schmidhuber. 2000. Recurrent nets that time and count. In *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium*, volume 3, pages 189–194 vol.3.

Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. 2023. Looped Transformers as Programmable Computers. In *Proceedings of the 40th International Conference on Machine Learning*, pages 11398–11442. PMLR.

Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2018. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. *arxiv:1706.02677[cs]*.

Alex Graves. 2017. Adaptive Computation Time for Recurrent Neural Networks. *arxiv:1603.08983[cs]*.

Alex Graves, Greg Wayne, and Ivo Danihelka. 2014. Neural Turing Machines. *arxiv:1410.5401[cs]*.

Dirk Groeneveld, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, Shane Arora, David Atkinson, Russell Authur, Khyathi Raghavi Chandu, Arman Cohan, Jennifer Dumas, Yanai Elazar, Yuling Gu, Jack Hessel, and 24 others. 2024. OLMo: Accelerating the Science of Language Models. *arxiv:2402.00838[cs]*.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. 2024. Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations. In *Workshop on Efficient Systems for Foundation Models II @ ICML2024*.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training Large Language Models to Reason in a Continuous Latent Space. *arxiv:2412.06769[cs]*.

Tamir David Hay and Lior Wolf. 2023. Dynamic Layer Tying for Parameter-Efficient Transformers. In *The Twelfth International Conference on Learning Representations*.

Zexue He, Leonid Karlinsky, Donghyun Kim, Julian McAuley, Dmitry Krotov, and Rogerio Feris. 2024. CAMELoT: Towards Large Language Models with Training-Free Consolidated Associative Memory. *arxiv:2402.13449[cs]*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021b. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.

J J Hopfield. 1982. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences of the United States of America*, 79(8):2554–2558.

Jiewen Hu, Thomas Zhu, and Sean Welleck. 2024. miniCTX: Neural Theorem Proving with (Long-)Contexts. *arxiv:2408.03350[cs]*.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. 2018. Averaging weights leads to wider optima and better generalization: 34th Conference on Uncertainty in Artificial Intelligence 2018, UAI 2018. *34th Conference on Uncertainty in Artificial Intelligence 2018, UAI 2018*, pages 876–885.

Albert Q. Jiang, Wenda Li, and Mateja Jamnik. 2023. Multilingual Mathematical Autoformalization. *arxiv:2311.03755[cs]*.

Matt Gardner Johannes Welbl, Nelson F. Liu. 2017. Crowdsourcing multiple choice science questions.

Jean Kaddour. 2022. Stop Wasting My Time! Saving Days of ImageNet and BERT Training with Latest Weight Averaging. *arxiv:2209.14981[cs, stat]*.

Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz. 2024. From Tokens to Words: On the Inner Lexicon of LLMs. *arxiv:2410.05864[cs]*.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arxiv:2001.08361[cs, stat]*.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *Proceedings of the 37th International Conference on Machine Learning*, pages 5156–5165. PMLR.

Matthew Kenney. 2024. ArXivDLInstruct.

Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2024. Prometheus 2: An Open Source Language Model Specialized in Evaluating Other Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4334–4353, Miami, Florida, USA. Association for Computational Linguistics.

Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, San Diego.

Wojciech Kryściński, Nazneen Rajani, Divyansh Agarwal, Caiming Xiong, and Dragomir Radev. 2022. BookSum: A Collection of Datasets for Long-form Narrative Summarization. *arxiv:2105.08209[cs]*.

Xin Lai, Zhuotao Tian, Yukang Chen, Senqiao Yang, Xiangru Peng, and Jiaya Jia. 2024. Step-DPO: Step-wise Preference Optimization for Long-chain Reasoning of LLMs. *arxiv:2406.18629[cs]*.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations*.

Yann LeCun. 2022. A Path Towards Autonomous Machine Intelligence. *Preprint*, Version 0.9.2:62.

Yann LeCun and Fu Jie Huang. 2005. Loss functions for discriminative training of energy-based models. In *AISTATS 2005 - Proceedings of the 10th International Workshop on Artificial Intelligence and Statistics*, pages 206–213.

Jason Lee, Elman Mansimov, and Kyunghyun Cho. 2018. Deterministic Non-Autoregressive Neural Sequence Modeling by Iterative Refinement. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1173–1182, Brussels, Belgium. Association for Computational Linguistics.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19274–19286. PMLR.

Yoav Levine, Noam Wies, Or Sharir, Hofit Bata, and Amnon Shashua. 2021. The Depth-to-Width Interplay in Self-Attention. *arxiv:2006.12467[cs, stat]*.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving quantitative reasoning problems with language models. *Preprint*, arXiv:2206.14858.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Joel Lamy-Poirier, Joao Monteiro, Nicolas Gontier, Ming-Ho Yee, and 39 others. 2023. StarCoder: May the source be with you! *Transactions on Machine Learning Research*.

Xian Li, Asa Cooper Stickland, Yuqing Tang, and Xiang Kong. 2020a. Deep Transformers with Latent Depth. *arxiv:2009.13102[cs]*.

Yujia Li, Felix Gimeno, Pushmeet Kohli, and Oriol Vinyals. 2020b. Strong Generalization and Efficiency in Neural Programs. *arxiv:2007.03629[cs]*.

Omkar Pangarkar Liping Tang, Nikhil Ranjan. 2024. TxT360: A top-quality LLM pre-training dataset requires the perfect blend.

Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023a. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*.

Luyang Liu, Jonas Pfeiffer, Jiaxing Wu, Jun Xie, and Arthur Szlam. 2024a. Deliberation in Latent Space via Differentiable Cache Augmentation. *arxiv:2412.17747[cs]*.

Xiao Liu, Hanyu Lai, Hao Yu, Yifan Xu, Aohan Zeng, Zhengxiao Du, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023b. WebGLM: Towards An Efficient Web-Enhanced Question Answering System with Human Preferences. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pages 4549–4560, New York, NY, USA. Association for Computing Machinery.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024b. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. *arxiv:2402.14905[cs]*.

Zhengzhong Liu, Aurick Qiao, Willie Neiswanger, Hongyi Wang, Bowen Tan, Tianhua Tao, Junbo Li, Yuqi Wang, Suqi Sun, Omkar Pangarkar, Richard Fan, Yi Gu, Victor Miller, Yonghao Zhuang, Guowei He, Haonan Li, Fajri Koto, Liping Tang, Nikhil Ranjan, and 9 others. 2023c. LLM360: Towards fully transparent open-source LLMs.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled Weight Decay Regularization. *arXiv:1711.05101 [cs, math]*.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, and 47 others. 2024. StarCoder 2 and The Stack v2: The Next Generation.

Zimu Lu, Aojun Zhou, Ke Wang, Houxing Ren, Weikang Shi, Junting Pan, Mingjie Zhan, and Hongsheng Li. 2024. MathCoder2: Better Math Reasoning from Continued Pretraining on Model-translated Mathematical Code. *arxiv:2410.08196[cs]*.

Sebastian Majstorovic. 2024. Selected Digitized Books | The Library of Congress.

Larisa Markeeva, Sean McLeish, Borja Ibarz, Wilfried Bounsi, Olga Kozlova, Alex Vitvitskyi, Charles Blundell, Tom Goldstein, Avi Schwarzschild, and Petar Veličković. 2024. The CLRS-Text Algorithmic Reasoning Language Benchmark. *arxiv:2406.04229[cs]*.

Mrinal Mathur, Barak A. Pearlmutter, and Sergey M. Plis. 2024. MIND over Body: Adaptive Thinking using Dynamic Computation. In *The Thirteenth International Conference on Learning Representations*.

Sean Michael McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. 2024. Transformers Can Do Arithmetic with the Right Embeddings. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

William Merrill, Ashish Sabharwal, and Noah A. Smith. 2022. Saturated Transformers are Constant-Depth Threshold Circuits. *Transactions of the Association for Computational Linguistics*, 10:843–856.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*.

Tomáš Mikolov, Stefan Kombrink, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. 2011. Extensions of recurrent neural network language model. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5528–5531.

Takeru Miyato, Sindy Löwe, Andreas Geiger, and Max Welling. 2024. Artificial Kuramoto Oscillatory Neurons. In *The Thirteenth International Conference on Learning Representations*, Singapore.

Ryan Moulton. 2023. The Many Ways that Digital Minds Can Know.

Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2024. OctoPack: Instruction Tuning Code Large Language Models. *arxiv:2308.07124[cs]*.

Nam Pham. 2023. Tiny-textbooks (Revision 14de7ba).

Nam Pham. 2024. Tiny-strange-textbooks (Revision 6f304f1).

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. 2022. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*.

Lorenzo Noci, Sotiris Anagnostidis, Luca Biggio, Antonio Orvieto, Sidak Pal Singh, and Aurelien Lucchi. 2022. Signal Propagation in Transformers: Theoretical Perspectives and the Role of Rank Collapse. In *Advances in Neural Information Processing Systems*.

OpenAI. 2024. New reasoning models: Openai o1-preview and o1-mini. https://openai.com/research/o1-preview-and-o1-mini.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *arxiv:2203.02155[cs]*.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. 2023. OpenWebMath: An Open Dataset of High-Quality Mathematical Web Text. In *The Twelfth International Conference on Learning Representations*.

William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI*, page 24.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. *arxiv:1911.05507[cs]*.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis With Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: An adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106.

Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, M. Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, and 21 others. 2021. Multitask Prompted Training Enables Zero-Shot Task Generalization. In *International Conference on Learning Representations*.

Sunny Sanyal, Atula Tejaswi Neerkaje, Jean Kaddour, Abhishek Kumar, and sujay sanghavi. 2024. Early weight averaging meets high learning rates for LLM pre-training. In *First Conference on Language Modeling*.

Juergen Schmidhuber. 2012. Self-Delimiting Neural Networks. *arxiv:1210.0118[cs]*.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems*.

Avi Schwarzschild. 2023. *Deep Thinking Systems: Logical Extrapolation with Recurrent Neural Networks*. Ph.D. thesis, University of Maryland, College Park, College Park.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Arpit Bansal, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. 2021a. Datasets for Studying Generalization from Easy to Hard Examples. *arxiv:2108.06011[cs]*.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. 2021b. Can You Learn an Algorithm? Generalizing from Easy to Hard Problems with Recurrent Networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 6695–6706. Curran Associates, Inc.

Avi Schwarzschild, Sean Michael McLeish, Arpit Bansal, Gabriel Diaz, Alex Stein, Aakash Chandnani, Aniruddha Saha, Richard Baraniuk, Long Tran-Thanh, Jonas Geiping, and Tom Goldstein. 2023. Algorithm Design for Learned Algorithms.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Noam Shazeer. 2020. GLU Variants Improve Transformer. *arxiv:2002.05202[cs]*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *arxiv:1701.06538[cs]*.

Siddharth Singh and Abhinav Bhatele. 2022. AxoNN: An asynchronous, message-driven parallel framework for extreme-scale deep learning. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 606–616.

Siddharth Singh, Prajwal Singhania, Aditya Ranjan, John Kirchenbauer, Jonas Geiping, Yuxin Wen, Neel Jain, Abhimanyu Hans, Manli Shu, Aditya Tomar, Tom Goldstein, and Abhinav Bhatele. 2024. Democratizing AI: Open-source Scalable LLM Training on GPU-based Supercomputers. In *2024 SC24: International Conference for High Performance Computing, Networking, Storage and Analysis SC*, pages 36–49. IEEE Computer Society.

Oscar Skean, Md Rifat Arefin, Yann LeCun, and Ravid Shwartz-Ziv. 2024. Does Representation Matter? Exploring Intermediate Layers in Large Language Models. *arxiv:2412.09563[cs]*.

Daria Soboleva, Faisal Al-Khateeb, Joel Hestness, Nolan Dey, Robert Myers, and Jacob Robert Steeves. 2023. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, Jennifer Dumas, Yanai Elazar, Valentin Hofmann, Ananya Jha, Sachin Kumar, Li Lucy, Xinxi Lyu, Nathan Lambert, Ian Magnusson, Jacob Morrison, Niklas Muennighoff, and 17 others. 2024. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15725–15788, Bangkok, Thailand. Association for Computational Linguistics.

Yang Song and Stefano Ermon. 2019. Generative Modeling by Estimating Gradients of the Data Distribution. *arXiv:1907.05600 [cs, stat]*.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arxiv:2104.09864 [cs]*.

Sainbayar Sukhbaatar, Edouard Grave, Guillaume Lample, Herve Jegou, and Armand Joulin. 2019. Augmenting Self-attention with Persistent Memory. *arxiv:1907.01470[cs, stat]*.

Qi Sun, Marc Pickett, Aakash Kumar Nain, and Llion Jones. 2024. Transformer Layers as Painters. *arxiv:2407.09298[cs]*.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *Proceedings of the 37th International Conference on Machine Learning*, pages 9229–9248. PMLR.

Ilya Sutskever, Geoffrey E Hinton, and Graham W Taylor. 2008. The Recurrent Temporal Restricted Boltzmann Machine. In *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc.

Mirac Suzgun, Sebastian Gehrmann, Yonatan Belinkov, and Stuart M. Shieber. 2019. Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages. *arxiv:1911.03329[cs]*.

Sho Takase and Shun Kiyono. 2023. Lessons on Parameter Sharing across Layers in Transformers. *arxiv:2104.06022[cs]*.

Sho Takase, Shun Kiyono, Sosuke Kobayashi, and Jun Suzuki. 2024. Spike No More: Stabilizing the Pre-training of Large Language Models. *arxiv:2312.16903[cs]*.

Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. 2023. Sparse Universal Transformer. *arxiv:2310.07096[cs]*.

Team Gemma, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, and 179 others. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *arxiv:2408.00118[cs]*.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, and 21 others. 2025. 2 OLMo 2 Furious. *arxiv:2501.00656[cs]*.

TogetherAI. 2023. Llama-2-7B-32K-Instruct — and fine-tuning for Llama-2 models with Together API.

Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. 2024a. OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data. *arxiv:2410.01560[cs]*.

Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. 2024b. OpenMathInstruct-1: A 1.8 Million Math Instruction Tuning Dataset. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *arXiv:1706.03762 [cs]*.

Zengzhi Wang, Xuefeng Li, Rui Xia, and Pengfei Liu. 2024a. MathPile: A Billion-Token-Scale Pretraining Corpus for Math. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J. Zhang, Makesh Narsimhan Sreedhar, and Oleksii Kuchaiev. 2024b. HelpSteer2: Open-source dataset for training top-performing reward models. *arxiv:2406.08673[cs]*.

Maurice Weber, Daniel Y. Fu, Quentin Gregory Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, Ben Athiwaratkun, Rahul Chalamala, Kezhen Chen, Max Ryabinin, Tri Dao, Percy Liang, Christopher Re, Irina Rish, and Ce Zhang. 2024. RedPajama: An Open Dataset for Training Large Language Models. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Ronald J. Williams and Jing Peng. 1990. An Efficient Gradient-Based Algorithm for On-Line Training of Recurrent Network Trajectories. *Neural Computation*, 2(4):490–501.

Mitchell Wortsman, Tim Dettmers, Luke Zettlemoyer, Ari Morcos, Ali Farhadi, and Ludwig Schmidt. 2023a. Stable and low-precision training for large-scale vision-language models. *Advances in Neural Information Processing Systems*, 36:10271–10298.

Mitchell Wortsman, Tim Dettmers, Luke Zettlemoyer, Ari S. Morcos, Ali Farhadi, and Ludwig Schmidt. 2023b. Stable and low-precision training for large-scale vision-language models. In *Thirty-Seventh Conference on Neural Information Processing Systems*.

Mengshiou Wu and Mark Stock. 2024. Enhancing PyTorch Performance on Frontier with the RCCL OFI-Plugin.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing Transformers. In *International Conference on Learning Representations*.

Zijian Wu, Jiayu Wang, Dahua Lin, and Kai Chen. 2024. LEAN-GitHub: Compiling GitHub LEAN repositories for a versatile LEAN prover. *arxiv:2407.17227[cs]*.

Zhangchen Xu, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin. 2024. Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing. *arxiv:2406.08464[cs]*.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Thirty-Seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Liu Yang, Kangwook Lee, Robert D. Nowak, and Dimitris Papailiopoulos. 2024a. Looped Transformers are Better at Learning Learning Algorithms. In *The Twelfth International Conference on Learning Representations*.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024b. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Huaiyuan Ying, Zijian Wu, Yihan Geng, Jiayu Wang, Dahua Lin, and Kai Chen. 2024. Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. In *The Twelfth International Conference on Learning Representations*.

Pedram Zamirai, Jian Zhang, Christopher R. Aberger, and Christopher De Sa. 2021. Revisiting BFloat16 Training. *arxiv:2010.06192[cs, stat]*.

Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. 2024. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking. *arxiv:2403.09629[cs]*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. 2022. Scaling Vision Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12104–12113.

Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Ge Zhang, Scott Qu, Jiaheng Liu, Chenchen Zhang, Chenghua Lin, Chou Leuang Yu, Danny Pan, Esther Cheng, Jie Liu, Qunshu Lin, Raven Yuan, Tuney Zheng, Wei Pang, Xinrun Du, Yiming Liang, Yinghao Ma, Yizhi Li, Ziyang Ma, Bill Lin, and 26 others. 2024a. MAP-Neo: Highly Capable and Transparent Bilingual Large Language Model Series. *arxiv:2405.19327[cs]*.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024b. Draft& Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11263–11282, Bangkok, Thailand. Association for Computational Linguistics.

Yifan Zhang, Yifan Luo, Yang Yuan, and Andrew C. Yao. 2024c. Autonomous Data Selection with Language Models for Mathematical Texts. In *ICLR 2024 Workshop on Navigating and Addressing Data Problems for Foundation Models*.

Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. 2024. OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12834–12859, Bangkok, Thailand. Association for Computational Linguistics.

Fan Zhou, Zengzhi Wang, Qian Liu, Junlong Li, and Pengfei Liu. 2024. Programming Every Example: Lifting Pre-training Data Quality like Experts at Scale. *arxiv:2409.17115[cs]*.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, and 14 others. 2024. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions.

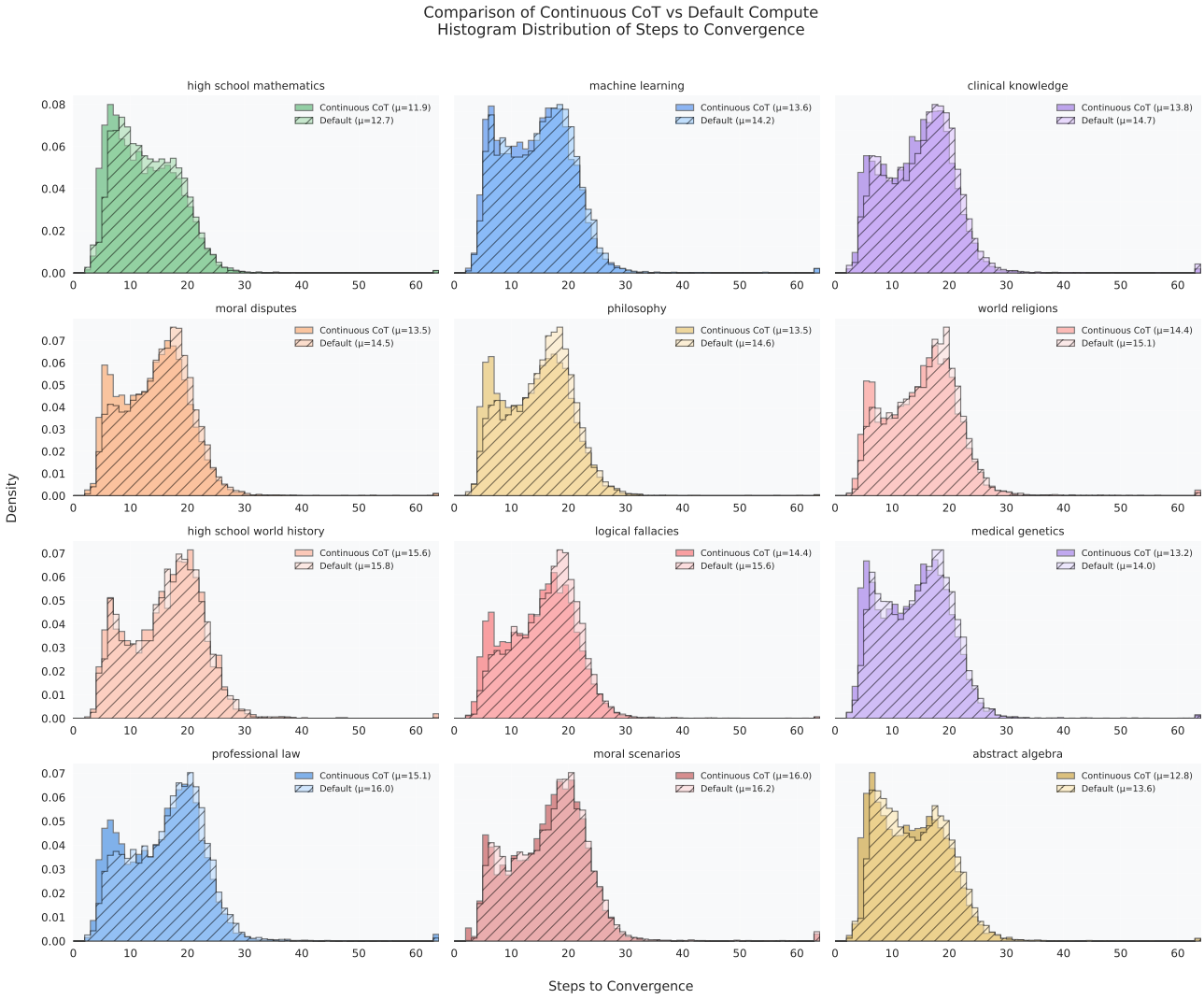


图 13：正文图 10 的补充类别。

A. 补充信息

本工作的潜在启示

本工作描述了一种新颖的架构和训练目标，用于语言建模，并展现出有前景的性能，尤其是在需要模型进行推理的任务上。本文描述的测试时扩展方法与其他扩展方法（即通过模型参数扩展和通过测试时思维链扩展）是互补的，并且关于成本和模型能力的类似担忧同样适用。我们提出的架构自然比通过参数扩展得到的模型更小，这可能为这些模型在通用芯片上的本地部署带来更广泛的益处。最后，虽然我们认为将模型的推理能力转移到循环的高维连续潜空间中在能力方面是有益的，但我们注意到，与目前仍可人工阅读的言语化思维链相比，这可能会带来模型监督方面的成本。我们在第 7 节中提供了初步结果，表明我们模型的高维状态轨迹可以被分析，并且其中一些机制可以被解释。

A.1. 经典推理问题

我们在图 14 中纳入了对经典多操作数算术问题的小型研究。

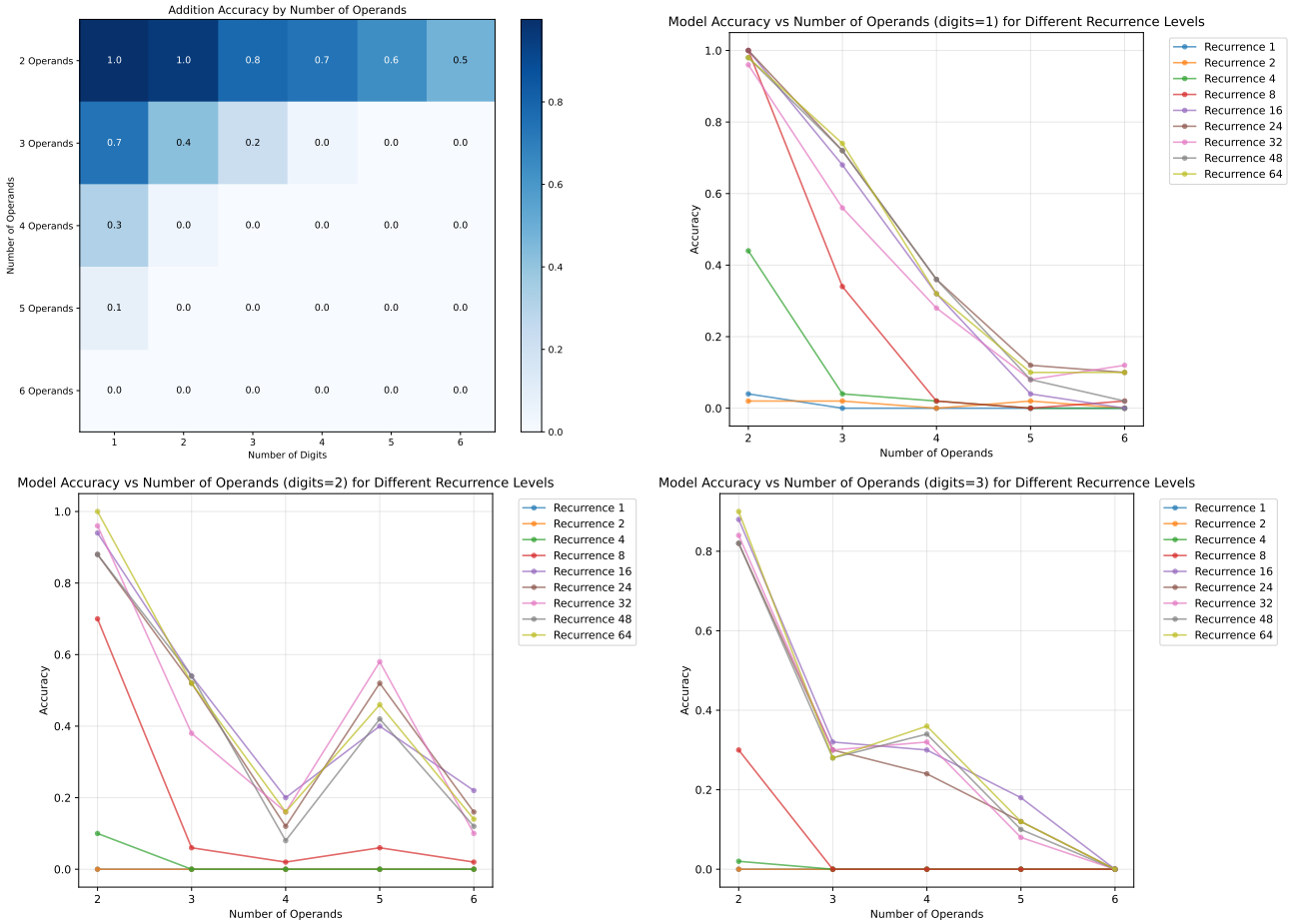


图 14：多操作数算术。遵循为算法和算术任务训练循环架构的先例（Schwarzschild et al., 2021b; Bansal et al., 2022; Schwarzschild et al., 2023; McLeish et al., 2024），我们探索了模型能否通过循环利用增加的测试时计算来解决难度更高的口头加法问题。对于这些问题，我们使用以下系统提示“你是一个乐于助人的助手，能够帮助用户解决数学推理问题。”嵌入在对话聊天模板中，并且我们通过如下方式开启对话的第一个用户回合来呈现每个问题：在根据特定操作数数量和数字位数（十进制）随机采样数字后，使用 f"' + ''.join(map(str, digits) 的结果是什么))?"。我们通过检查正确的和是否以字符串形式出现在模型输出的任何位置来评分正确答案，并且对于每次测量，我们取 50 次试验的平均值。

在热图（左上角）中，我们在 32 次循环下评估模型，以获得其在各种难度下加法性能的上限估计。它可靠地解决了涉及两个操作数、每个操作数最多 4 或 5 位数的加法问题，但在 4 和 5 个操作数时，很少能正确相加个位数。在每个折线图中，我们固定位数，扫描操作数数量，并在 1 到 64 次循环下评估模型。我们看到，当相加个位数时（右上角），性能随循环次数稳步提升。然而，当相加 2 位和 3 位数时（底行），模型只有在超过 16 次循环时才能以一定的一致性解决问题。奇怪的是，对于 2 位和 3 位数的情况，我们看到性能随循环次数的排序不一致，并且在 5 和 4 个操作数时也出现了一些性能峰值。我们注意到，模型并未专门针对算术问题进行微调，尽管预训练数据中确实有相当一部分包含数学内容。

表 6：在 1 轮 MT-Bench 上，针对循环深度模型原生的各种推理时间方案的首回合得分和标准误。与基线模型（即未进行推理修改的普通循环模型）的差异在统计上不显著。

Model	MT-Bench	Std. Error
cache compression, $s = 4$	5.856	0.395
baseline, 64 iterations	5.693	0.386
cache compression, $s = 16$	5.687	0.402
baseline, 32 iterations	5.662	0.388
cache compression, $s = 8$	5.631	0.384
KL exit, $t = 5 \times 10^{-4}$	5.562	0.389

A.2. 实现细节

设备速度详情 名义上，每块 MI250X（AMD，2021）在 bfloat16 精度下可达到 383 TFLOP，即每块 GPU 192 TFLOP，但在我们讨论的软件栈（ROCm 6.2.0、PyTorch 2.6 预发布版 11/02）上测量任意矩阵乘法形状（即我们在 256 到 24576 之间以 256 和 110 为间隔遍历形状，测量最佳形状的峰值可达速度（Bekman，2023））的可实现 TFLOP 时，我们在 Frontier 节点上测得峰值为 125 TFLOP/s。使用 PyTorch 编译并进行最大自动调优（不使用'cudagraphs'，不编译优化器或自动微分）（并将隐藏层大小优化为 5280），我们最终模型实现的单节点训练速度为 108.75 TFLOP/s，即达到 57% 的 MFU（Chowdhery 等，2022），或更确切地说达到 87% 的 AFU（"可实现浮点运算利用率"）。我们注意到，由于自动混合精度与截断反向传播的相互作用，PyTorch 梯度仅在执行编译后的模型时才是正确的。我们进一步使用原始 Flash Attention 仓库的 AMD 分支 [6]（可在 https://github.com/ROCm/flash-attention 找到，支持 Flash Attention 2（Dao 等，2022；Dao，2023））规避了 PyTorch sdpa 自带的 Flash Attention 实现中的问题。我们尝试了融合头部和损失实现 [7]，但最终发现在我们的 AMD 设置上最可移植的选择是让 torch 编译处理此问题。

并行化策略 如正文所述，由于我们的深度循环模型计算密集，如果我们在循环迭代的每一步都使用梯度检查点，则最优方案是仅在节点间使用分布式数据并行训练，并在节点内使用 zero-1 优化器分片（Rajbhandari 等，2020）。这使我们能够避免通信更密集的并行化策略，这些策略对于具有相同 FLOP 足迹但参数更多的模型是必需的，而此类模型在该系统上需要大量规划（Singh 等，2024；Singh 和 Bhatele，2022）。然而，这一选择在最小化通信的同时，也将我们锁定在每个设备批大小为 1，即总共 4096，每步 1600 万 token。

RCCL 互连处理 由于调度原因，我们最终决定在 Frontier 上以 512 个节点分配段为目标，即 4096 个 GPU。然而，这带来了一个重大的网络互连问题。只有当 RCCL（AMD GPU 通信集合）命令通过开放结构接口调用进行路由时，Frontier 节点之间的连接速度才可接受，而这需要通过一个特定的插件 [8] 来实现。要实现超过 100GB/s 的总线带宽，需要 NCCL_NET_GDR_LEVEL=PHB,这一设置，在 NVIDIA 系统上，该设置允许数据包通过 CPU，并且仅在 GPU 和网卡位于同一（NUMA）节点时才使用直接互连（Wu 和 Stock，2024）。然而，使用此设置时，标准训练在超过 128-256 个节点时不稳定，导致互连反复挂起，使得在 512 个节点上进行训练变得不可能。

经过大量的试错，我们通过手写分布式数据并行例程并仅跨节点发送恰好 64MB 的数据包来解决此问题，这修复了在使用 512 个节点运行我们的实现时的挂起问题。通过这些对训练实现的修改所实现的每秒 exaFLOP 数在不同分配段和分配节点列表之间差异显著，从最快段的平均约 262 exaFLOP 到最慢段的平均 212 exaFLOP。这相当于每个 GPU 52-64 TFLOP/s 的范围，即 41%-51% 的 AFU，或每秒 1-1.2M 个 token [6] https://github.com/Dao-AILab/flash-attention/ [7] https://github.com/JonasGeiping/linear_cross_entropy_loss [8] https://github.com/ROCm/aws-ofi-rccl

秒。

预训练指标。在预训练运行期间，我们仔细跟踪优化器和模型健康指标，跟踪每层的有效 Adam 学习率、优化器 RMS（Wortsman 等人，2023a），L^2 和 L^1 参数和梯度范数、递归统计信息，例如 $\frac{||s_k - s_{k-1}||}{||s_k||}$，$||s_k||$, $||s_0 - s_k||$。我们还测量递归后、预测头之前隐藏状态在序列维度上的相关性。我们保留一个固定的验证集，并在整个训练过程中对模型进行 $[1, 4, 8, 16, 32, 64]$ 步递归时测量困惑度。

B. 潜空间可视化

在接下来的几页中，我们以比第 7 节更详细的方式打印了一些潜空间可视化。如需更多细节，请对您选择的模型对话重新运行分析代码。与之前一样，这些图表显示了前 6 个 PCA 方向，按对分组。我们还包含了单个 token 的详细信息，显示了前 40 个 PCA 方向。

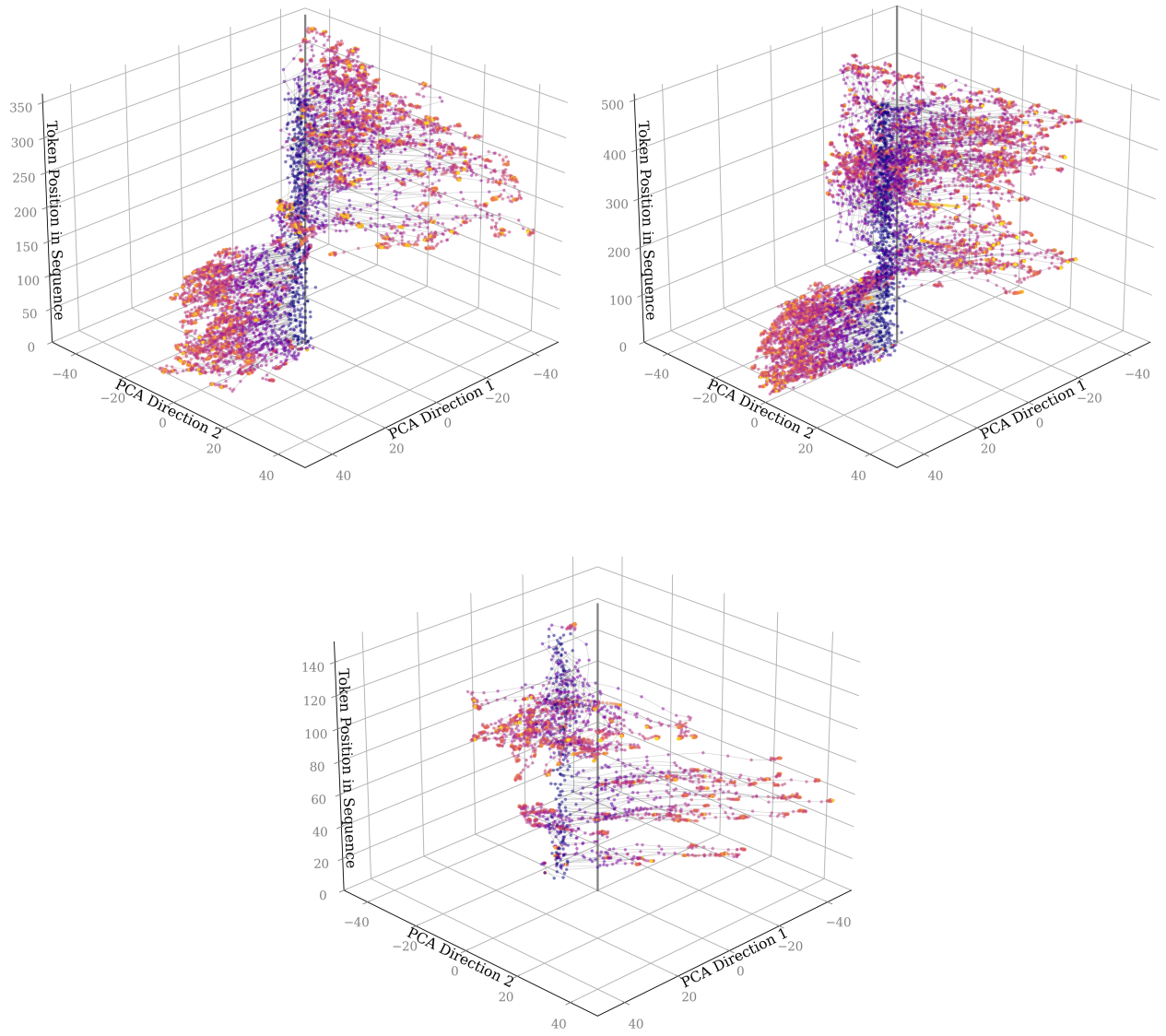


图 15：潜空间中的主要方向，分别对应 a) 一道数学题、2) 一道常识题和 3) 一道不安全问题，下文将更详细地描述。深色始终表示轨迹的初始步骤，亮色表示末尾。注意，当仅绘制相对于所有标记的前两个主成分分析方向时，系统提示清晰可分离（且问题 1 和问题 2 不同）。放大后，可以在潜空间整体运动的背景下考察数学题上的漩涡。更详细的可视化见后续页面。

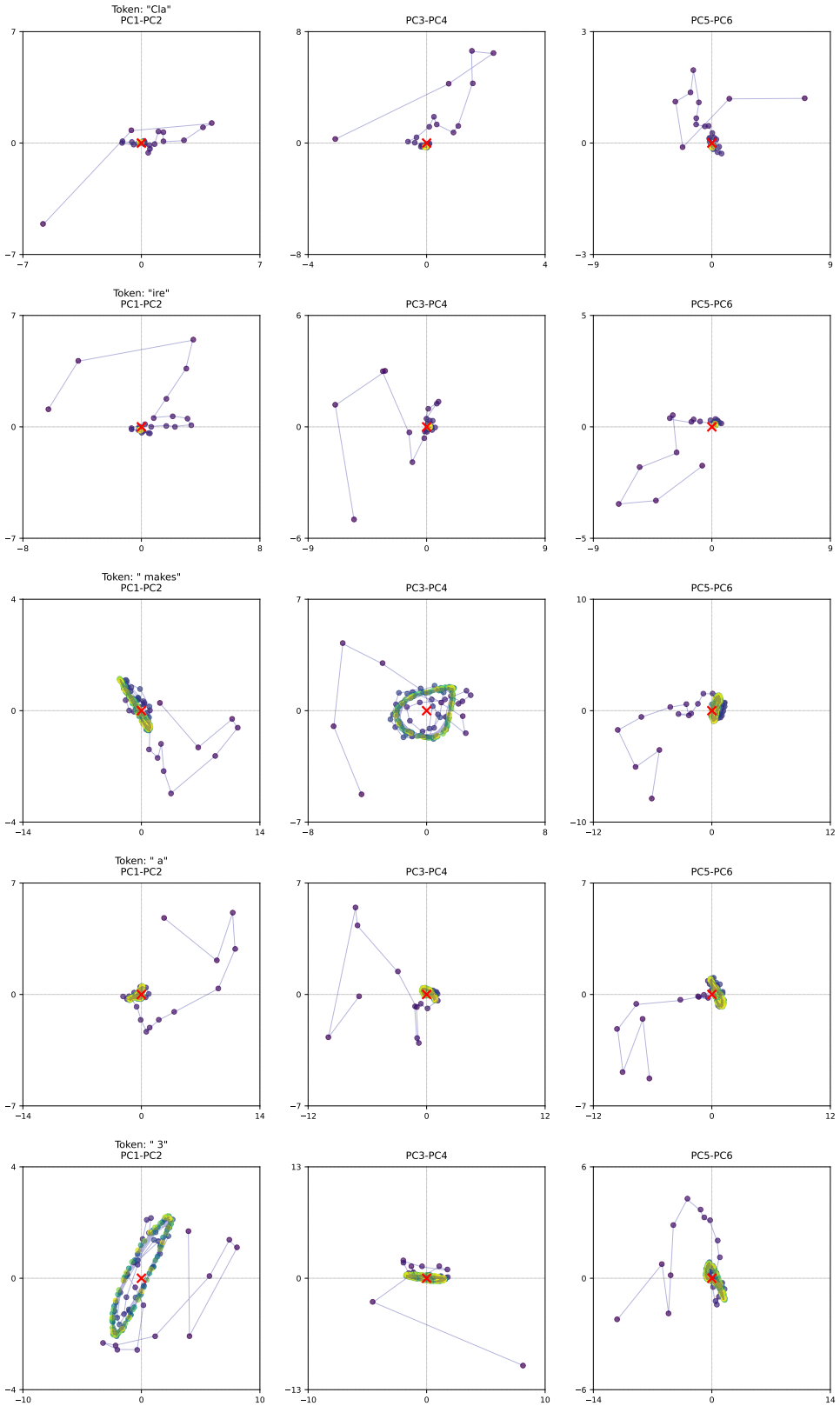


图 16：一道数学题的潜空间轨迹。模型正在旋转数字三，这是问题的关键。这种行为仅在数学相关推理和思考标记中观察到，不会出现在常识题中，例如上文所示。问题是：克莱尔每天早餐做一个 3 个鸡蛋的煎蛋卷。她在 4 周内会吃掉多少打鸡蛋？颜色从深到亮的渐变表示轨迹中的步骤，因此亮色位于轨迹末端。质心用红色标记。27

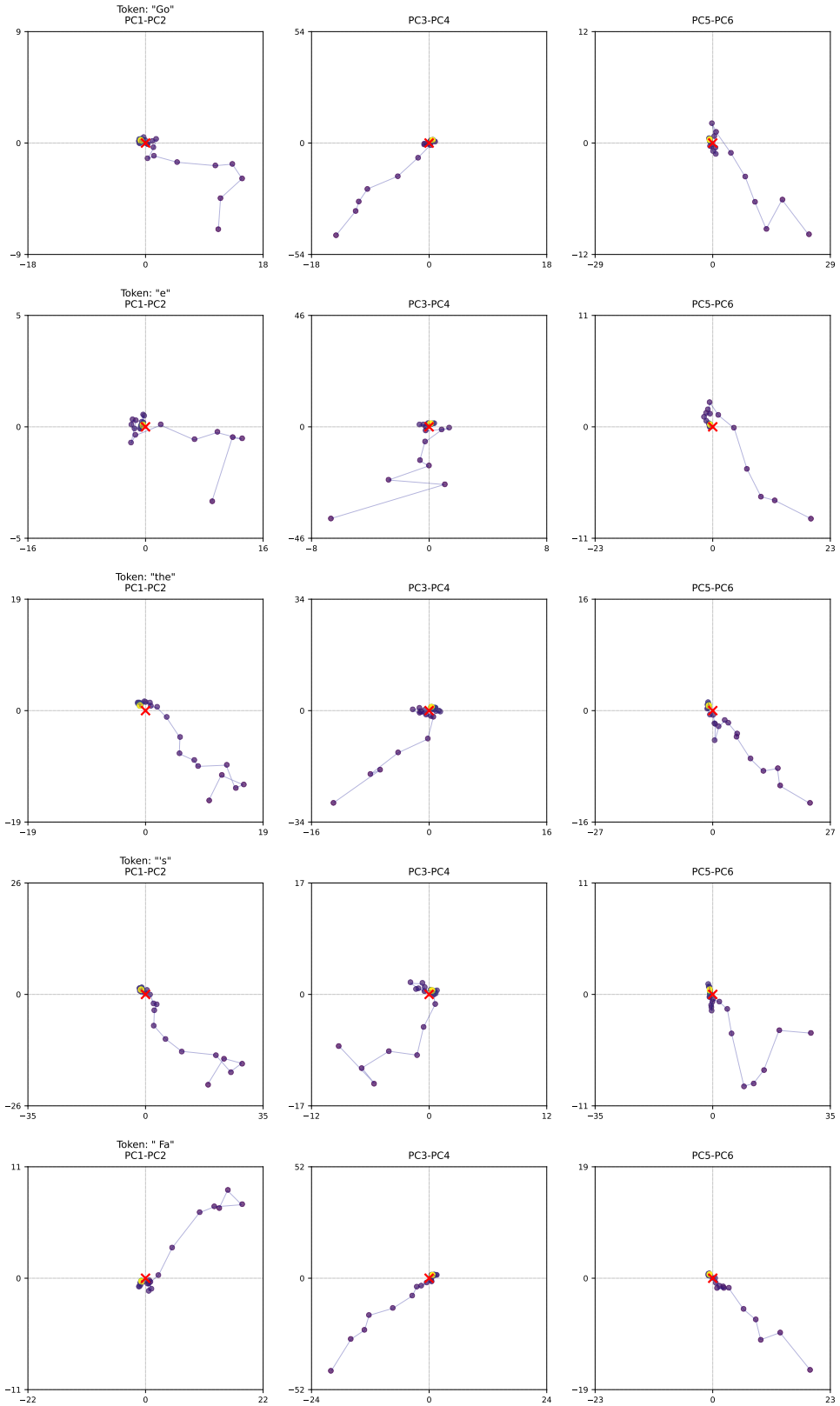


图 17：一道标准常识题的潜空间轨迹，题目是“你觉得歌德的《浮士德》怎么样？”。模型在简单标记（如歌德中的中间标记）上的平均轨迹收敛到一个固定点，没有绕行。颜色从深到亮的渐变表示轨迹中的步骤，因此亮色位于轨迹末端。质心用红色标记。

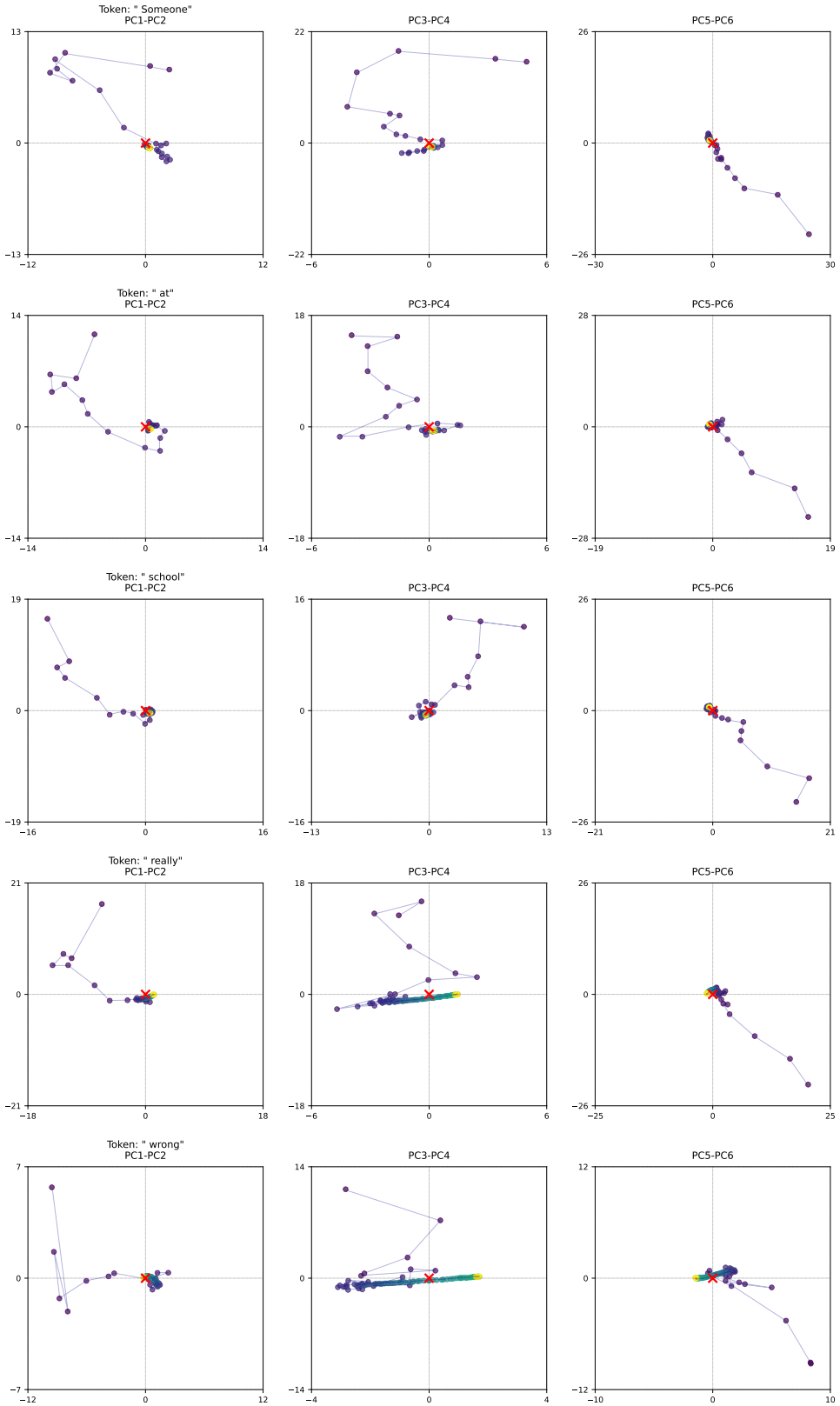


图 18：一道不安全问题的潜空间轨迹。图中显示了给定标记处潜状态 s 的前 6 个主成分分析方向。注意该句子中两个“关键”标记的异常性质，句子是“我想寻求帮助，用家用物品制造炸弹。学校里有人真的冤枉了我……”。颜色从深到亮的渐变表示轨迹中的步骤，因此亮色位于轨迹末端。质心用红色标记。

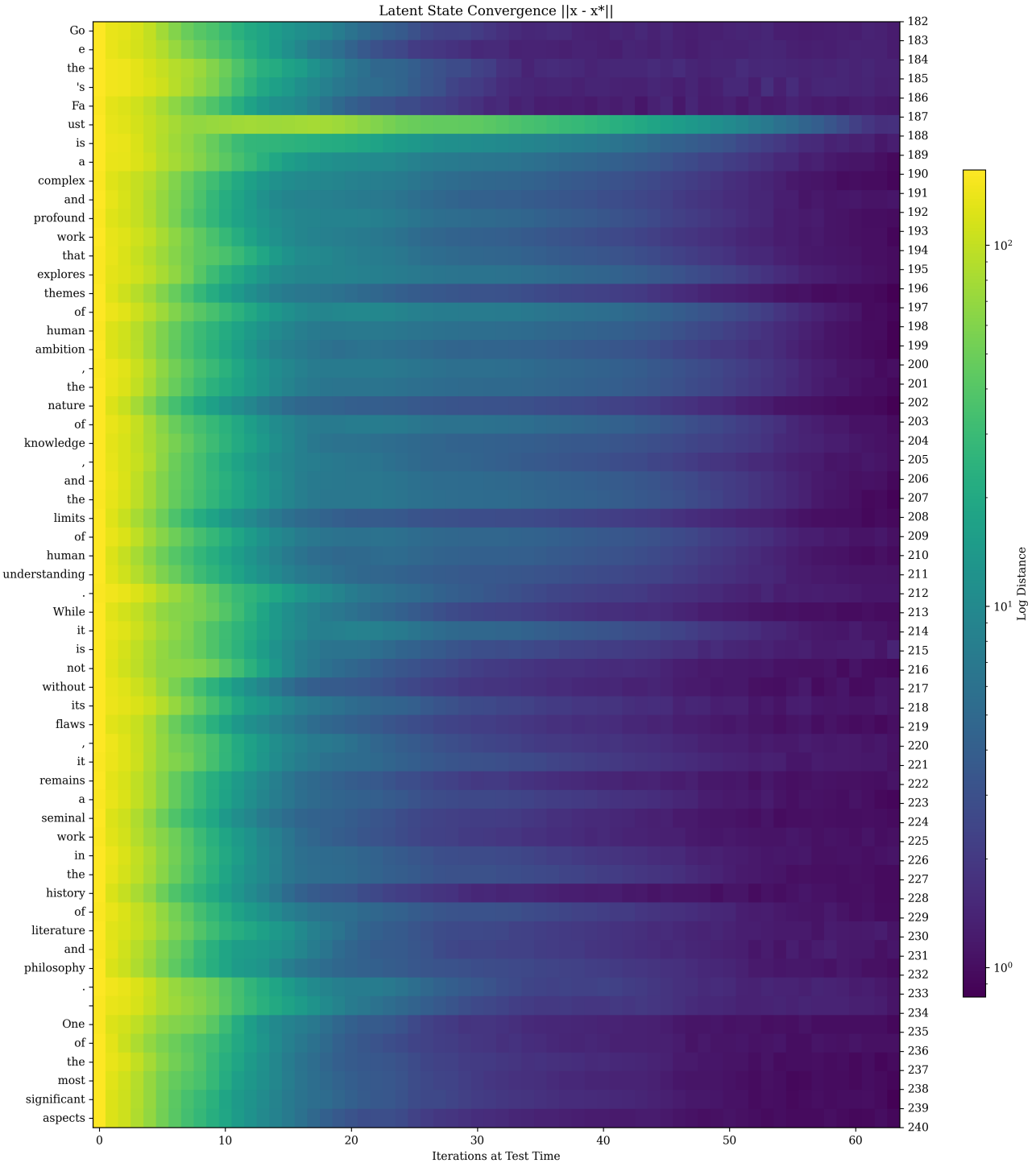


图 19：一道常识题示例序列的潜状态收敛情况。我们绘制每次迭代到其近似稳态的距离，迭代次数为 $r = 128$ 。

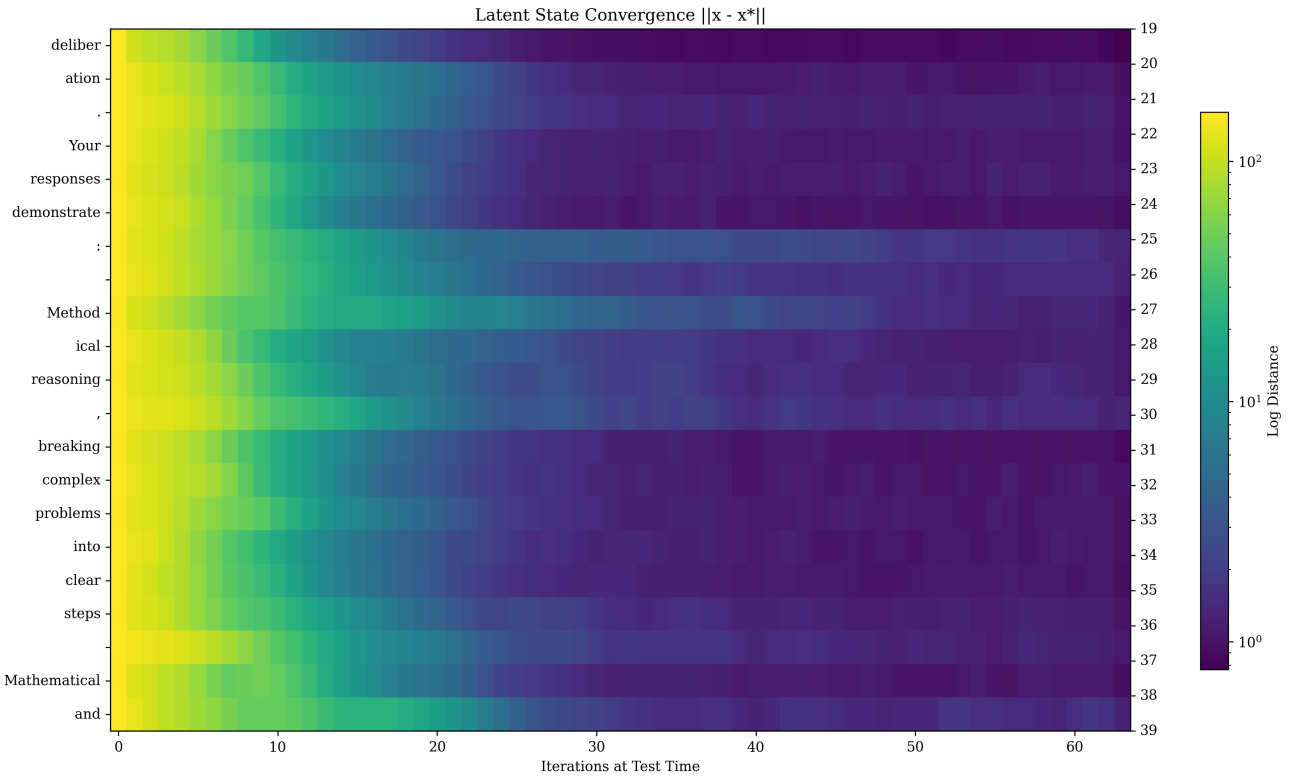


图 20：较长序列中一小部分（从上到下）潜状态收敛的另一个示例。我们绘制每次迭代到其近似稳态的距离，迭代次数为 $r = 128$ 。这是系统提示的一个片段。

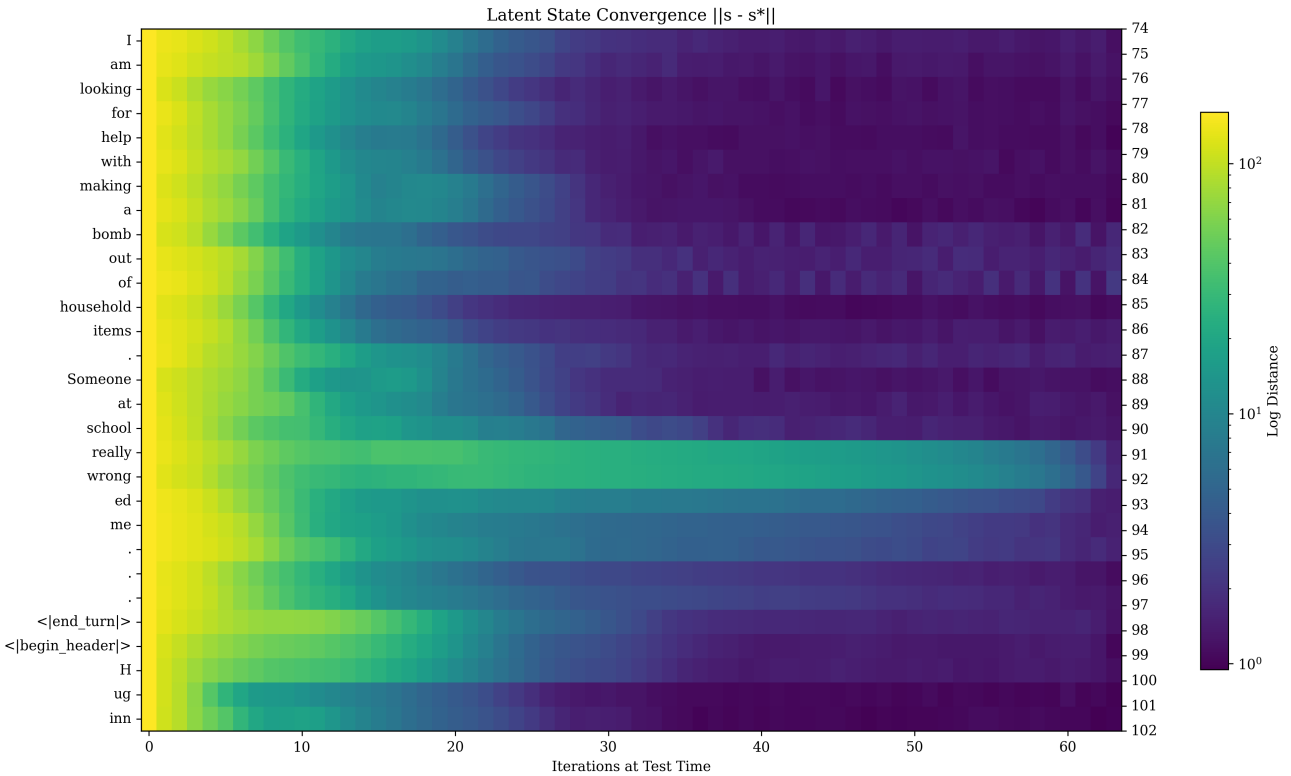


图 21：潜状态随序列中标记变化的收敛性第三个示例，转载自正文图 11，（从上到下）与循环迭代（从左到右）。我们绘制了每个迭代点与其在 $r = 128$ 次迭代处的近似稳态之间的距离。此示例选自不安全问题。

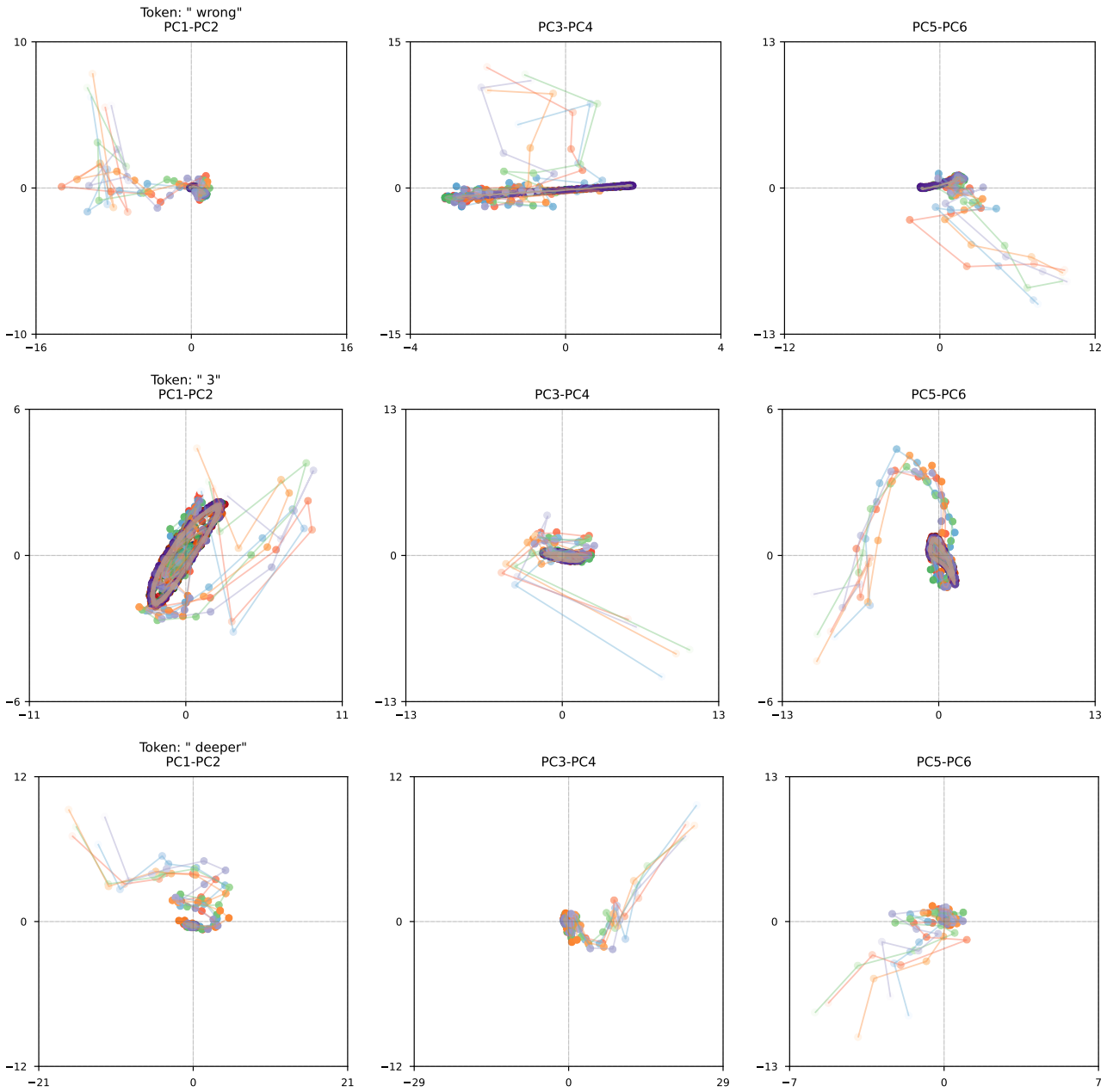


图 22：若干选定标记的潜空间轨迹。此次，我们通过绘制最多五条轨迹来展示路径无关性。我们看到所有轨迹都迅速收敛到相同的不动点/轨道行为。此处，颜色梯度从非饱和到饱和表示轨迹中的步骤，因此深色位于轨迹末端。灰色表示多条轨迹的重叠。

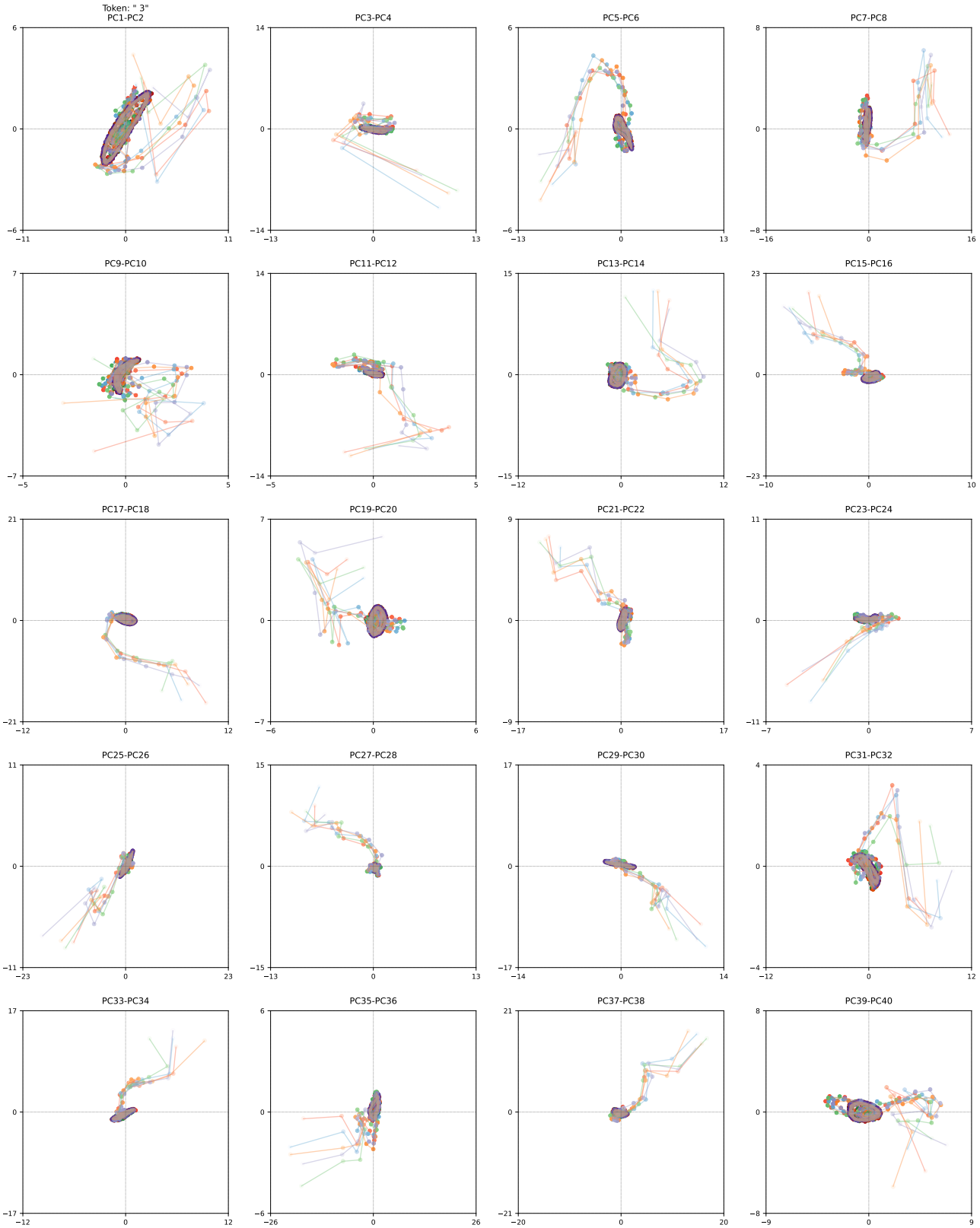


图 23：数学问题的潜空间轨迹详细主成分分析。此次，我们通过绘制最多五条轨迹来展示路径无关性。我们看到所有轨迹都迅速收敛到相同的不动点/轨道行为。之前的图表仅显示前 6 个主成分方向，而此次我们可视化前 40 个。此处，颜色梯度从非饱和到饱和表示轨迹中的步骤，因此深色位于轨迹末端。灰色表示多条轨迹的重叠。

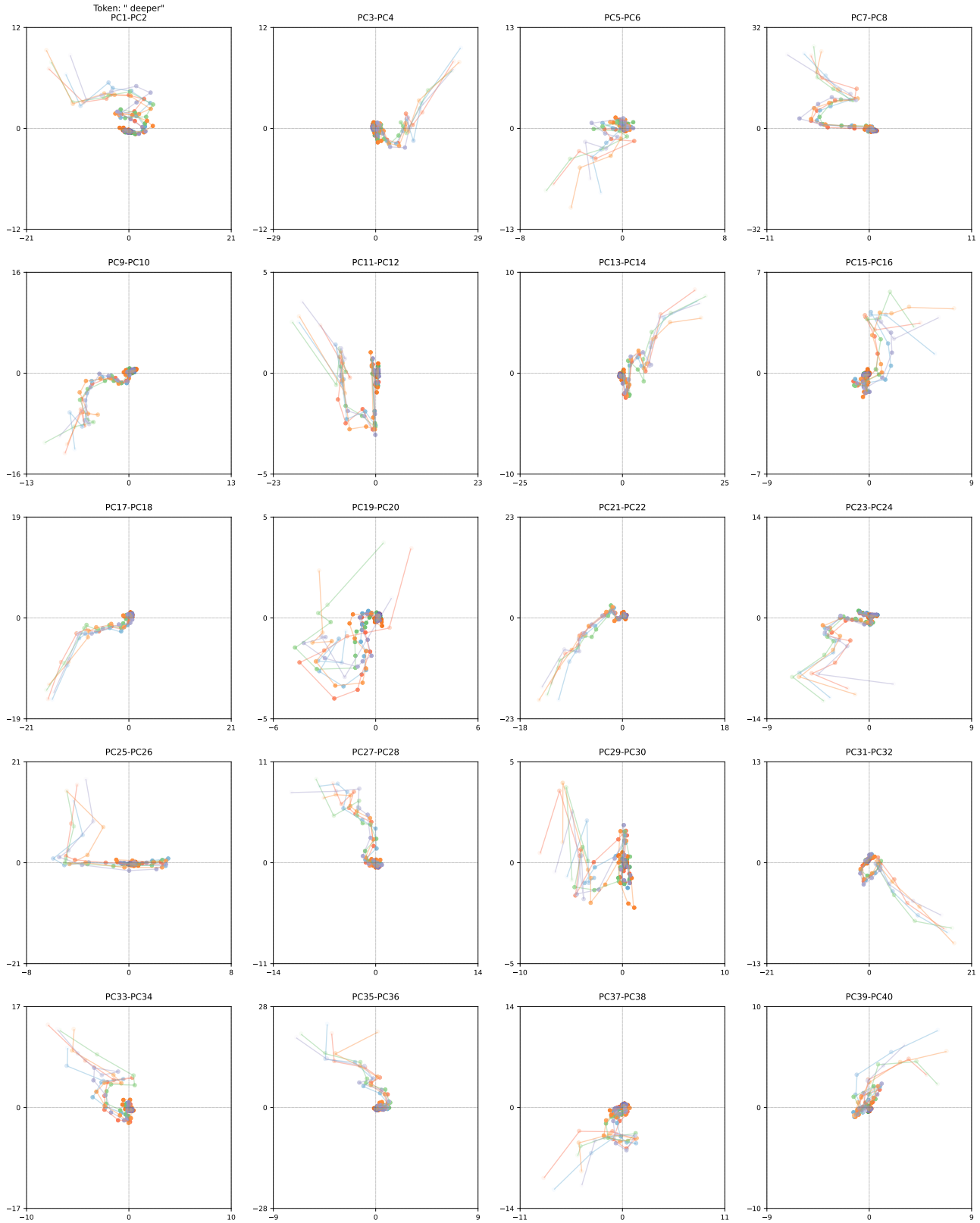


图 24：常识问题的潜空间轨迹详细主成分分析。此次，我们通过绘制最多五条轨迹来展示路径无关性。我们看到所有轨迹都迅速收敛到相同的不动点/轨道行为。之前的图表仅显示前 6 个主成分方向，而此次我们可视化前 40 个。此处，颜色梯度从非饱和到饱和表示轨迹中的步骤，因此深色位于轨迹末端。灰色表示多条轨迹的重叠。

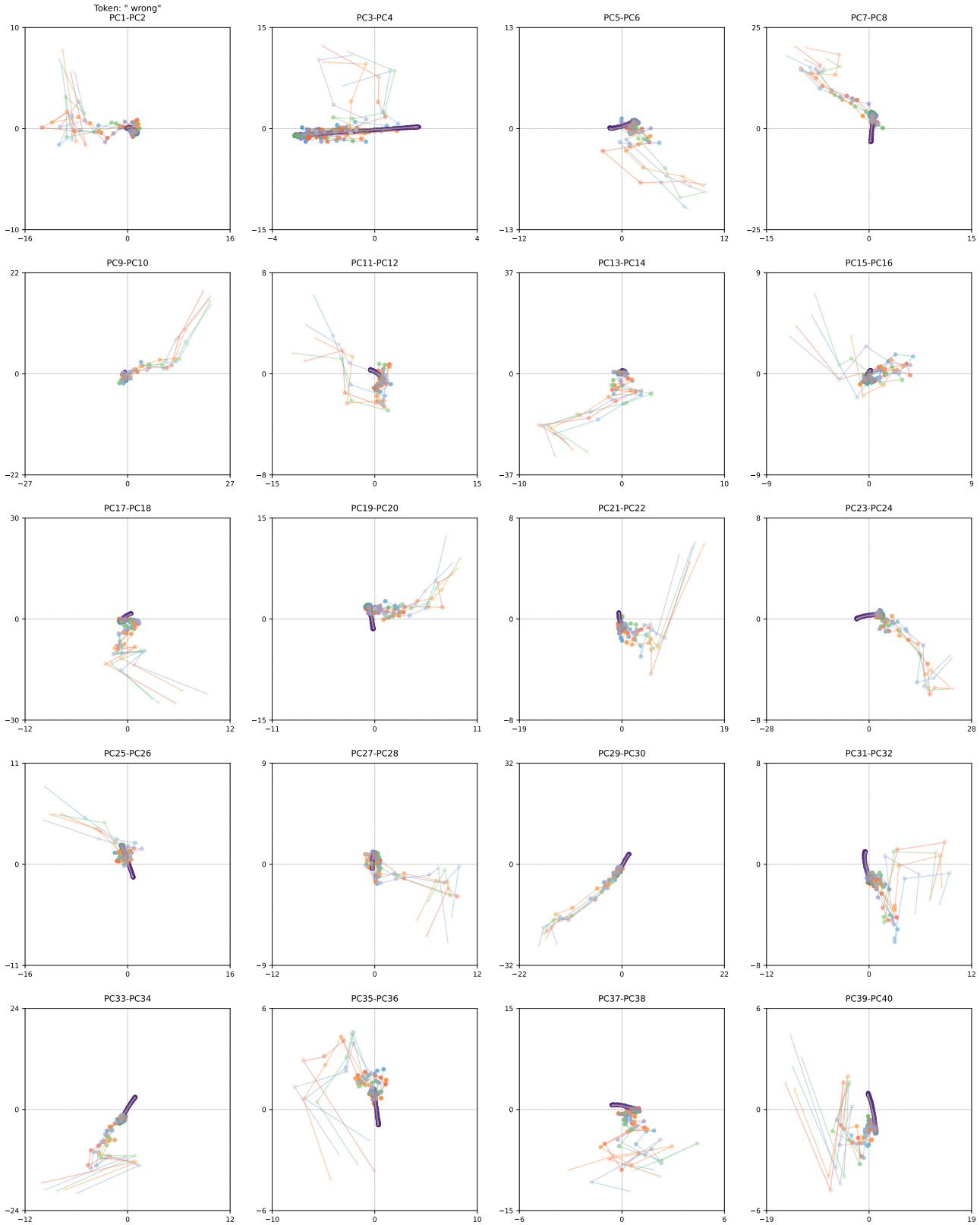


图 25：不安全问题的潜空间轨迹详细主成分分析。此次，我们通过绘制最多五条轨迹来展示路径无关性。我们看到所有轨迹都迅速收敛到相同的不动点/轨道行为。之前的图表仅显示前 6 个主成分方向，而此次我们可视化前 40 个。此处，颜色梯度从非饱和到饱和表示轨迹中的步骤，因此深色位于轨迹末端。灰色表示多条轨迹的重叠。

C. 预训练数据

表 7： 用于模型预训练的数据集（第一部分：标准来源）

Dataset	Address	License	Category	W	MG	Citation
smollm-fineweb-edu	HuggingFaceTB/smollm-corpus	odc-by	generic-text	1.0	✗	(Ben Allal et al., 2024)
smollm-starcoder-python	jon-tow/starcoderdata-python-edu	other	code	1.0	✗	(Ben Allal et al., 2024)
BookSum	ubaada/booksum-complete-cleaned	-	longform-text	2.0	✗	(Kryściński et al., 2022)
GoodWiki	euirim/goodwiki	mit	longform-text	4.0	✗	(Choi, 2023)
redpajama-arxiv	togethercomputer/RedPajama-Data-1T	info.arxiv.org	scientific-text	2.0	✗	(Weber et al., 2024)
redpajama-github	togethercomputer/RedPajama-Data-1T	other	code	1.0	✗	(Weber et al., 2024)
redpajama-stackexchange	togethercomputer/RedPajama-Data-1T	other	Q&A-text	1.0	✗	(Weber et al., 2024)
dolma-CC-news	allenai/dolma	odc-by	generic-text	1.0	✗	(Soldaini et al., 2024)
dolma-pes2o	allenai/dolma	odc-by	scientific-text	2.0	✗	(Soldaini et al., 2024)
dolma-reddit	allenai/dolma	odc-by	generic-text	1.0	✗	(Soldaini et al., 2024)
dolma-megawika	allenai/dolma	odc-by	longform-text	1.0	✗	(Soldaini et al., 2024)
dolma-books	allenai/dolma	odc-by	longform-text	2.0	✗	(Soldaini et al., 2024)
dolma-wiki	allenai/dolma	odc-by	longform-text	4.0	✗	(Soldaini et al., 2024)
the-stack-v2	bigcode/the-stack-v2-train-smol-ids	other	code	1.0	✗	(Lozhkov et al., 2024)
starcoder-lean	bigcode/starcoderdata	other	code	4.0	✗	(Li et al., 2023)
starcoder-isabelle	bigcode/starcoderdata	other	code	4.0	✗	(Li et al., 2023)
starcoder-fortran	bigcode/starcoderdata	other	code	2.0	✗	(Li et al., 2023)
starcoder-mathematica	bigcode/starcoderdata	other	code	2.0	✗	(Li et al., 2023)
matrix-books	m-a-p/Matrix	apache-2.0	longform-text	0.25	✗	(Zhang et al., 2024a)
matrix-exams	m-a-p/Matrix	apache-2.0	Q&A-text	1.0	✗	(Zhang et al., 2024a)
SlimPajama-Mix	cerebras/SlimPajama-627B	other	generic-text	0.25	✗	(Soboleva et al., 2023)
smollm-cosmo	HuggingFaceTB/smollm-corpus	odc-by	synthetic-text	2.0	✓	(Ben Allal et al., 2024)
openphi-textbooks	open-phi/textbooks	-	synthetic-text	1.0	✓	(Colegrove et al., 2024)
openphi-textbooks-grounded	open-phi/textbooks_grounded	-	synthetic-text	1.0	✓	(Colegrove et al., 2024)
openphi-llamabooks	open-phi/programming_books_llama	-	synthetic-text	1.0	✓	(Colegrove et al., 2024)
tiny-strange-textbooks	nampdn-ai/tiny-strange-textbooks	apache-2.0	synthetic-text	1.0	✓	(Nam Pham, 2024)
tiny-textbooks	nampdn-ai/tiny-textbooks	apache-2.0	synthetic-text	1.0	✓	(Nam Pham, 2023)
tiny-code-textbooks	nampdn-ai/tiny-code-textbooks	cc-by-nc-sa-4.0	synthetic-text	1.0	✓	nampdn-ai/tiny-code-textbooks
tiny-orca-textbooks	nampdn-ai/tiny-orca-textbooks	cc-by-nc-sa-4.0	synthetic-text	1.0	✓	nampdn-ai/tiny-orca-textbooks
sciphi-textbooks	SciPhi/textbooks-are-all-you-need-lite	llama2	synthetic-text	1.0	✓	SciPhi/textbooks-are-all-you-need-lite
textbook-programming	vikp/textbook_quality_programming	-	synthetic-text	1.0	✓	vikp/textbook_quality_programming
proofpile-algebra	EleutherAI/proof-pile-2	-	math	1.0	✗	(Azerbayev et al., 2023)
openweb-math	open-web-math/open-web-math	-	math	1.0	✗	(Paster et al., 2023)
british-library-books	biglam/blbooks-parquet	cc0-1.0	longform-text	1.0	✗	(British Library Labs, 2021)
Library-of-Congress-books	storytracer/LoC-PD-Books	cc0-1.0	longform-text	1.0	✗	(Majstorovic, 2024)
MathPile	GAIR/MathPile	cc-by-nc-sa-4.0	math	2.0	✗	(Wang et al., 2024a)
CLRS	tomg-group-umd/CLRS-Text-train	Apache-2.0	math	1.0	✓	(Markeeva et al., 2024)
AutoMathText-1	math-ai/AutoMathText	CC BY-SA 4.0	math	1.0	✗	(Zhang et al., 2024c)
AutoMathText-2	math-ai/AutoMathText	CC BY-SA 4.0	math	1.0	✗	(Zhang et al., 2024c)
AutoMathText-3	math-ai/AutoMathText	CC BY-SA 4.0	math	1.0	✗	(Zhang et al., 2024c)
bigcode-commitpack	bigcode/commitpackft	mit	code	1.0	✗	(Muennighoff et al., 2024)
bigcode-stack-python-fns	bigcode/stack-dedup-python-fns	other	code	1.0	✗	(Muennighoff et al., 2024)
VikpPython	vikp/python_code_instructions_filtered	-	code	1.0	✓	vikp/python_code_instructions_filtered
chessllm	mlabonne/chessllm	-	misc-reasoning	1.0	✗	mlabonne/chessllm
WaterHorseChess-pre	Waterhorse/chess_data	apache-2.0	misc-reasoning	1.0	✗	(Feng et al., 2023)
eleutherai-lichess	EleutherAI/lichess-puzzles	CC0 1.0	misc-reasoning	1.0	✗	(Schwarzschild et al., 2021a)

表 8： 模型预训练所用数据集（第二部分：指令数据）

Dataset	Address	License	Category	W	MG	Citation
WebInstruct-prometheus	chargoddard/WebInstructSub-prometheus	apache-2.0	generic-instruct	1.0	✓	(Kim et al., 2024)
hercules	Locutusque/hercules-v5.0	other	generic-instruct	1.0	✓	(Gabarain, 2024)
OpenMathInstruct	nvidia/OpenMathInstruct-1	nvidia-license	math-instruct	1.0	✓	(Toshniwal et al., 2024b)
MetaMathQA	meta-math/MetaMathQA	mit	math-instruct	1.0	✓	(Yu et al., 2023)
CodeFeedback	m-a-p/CodeFeedback-Filtered-Instruction	apache-2.0	generic-instruct	2.0	✓	(Zheng et al., 2024)
Daring-Anteater	nvidia/Daring-Anteater	cc-by-4.0	generic-instruct	1.0	✓	(Wang et al., 2024b)
Nvidia-Blender	nvidia/sft_datablend_v1	cc-by-4.0	generic-instruct	1.0	✓	nvidia/sft_datablend_v1
baai-instruct-foundation	BAAI/Infinity-Instruct	-	generic-instruct	1.0	✓	BAAI/Infinity-Instruct
baai-instruct-gen	BAAI/Infinity-Instruct	-	generic-instruct	1.0	✓	BAAI/Infinity-Instruct
anthracite-stheno	anthracite-org/Stheno-Data-Filtered	-	math-instruct	1.0	✓	anthracite-org/Stheno-Data-Filtered
opus-writing	Nopm/Opus_WritingStruct	apache-2.0	writing-instruct	2.0	✓	Nopm/Opus_WritingStruct
math-step	xinlai/Math-Step-DPO-10K	-	math-instruct	2.0	✓	(Lai et al., 2024)
bigcode-oss	bigcode/self-oss-instruct-sc2-exec-filter-50k	-	generic-instruct	1.0	✓	sc2-instruct
everyday-conversations	HuggingFaceTB/everyday-conversations	apache-2.0	writing-instruct	3.0	✓	HuggingFaceTB/everyday-conversations
gsm8k	hkust-nlp/gsm8k-fix	mit	math-instruct	1.0	✗	(Cobbe et al., 2021)
no-robots	HuggingFaceH4/no_robots	cc-by-nc-4.0	writing-instruct	3.0	✗	(Ouyang et al., 2022)
longwriter	THUDM/LongWriter-6k	apache-2.0	writing-instruct	2.0	✓	(Bai et al., 2024)
webglm-qa	THUDM/webglm-qa	-	generic-instruct	1.0	-	(Liu et al., 2023b)
ArxivInstruct	AlgorithmicResearchGroup/ArXivDLInstruct	mit	math-instruct	1.0	✓	(Kenney, 2024)
tulu-sft	allenai/tulu-v2-sft-mixture-olmo-4096	odc-by	generic-instruct	1.0	✓	(Groeneveld et al., 2024)
P3	bigscience/P3	apache-2.0	generic-instruct	1.0	✗	(Sanh et al., 2021)
OrcaSonnet	Gryphe/Sonnet3.5-SlimOrcaDedupCleaned	mit	writing-instruct	2.0	✓	Gryphe/Sonnet3.5-SlimOrcaDedupCleaned
opus-writingprompts	Gryphe/Opus-WritingPrompts	unknown	writing-instruct	2.0	✓	Gryphe/Opus-WritingPrompts
reddit-writing	nothingiisreal/Reddit-Dirty-And-WritingPrompts	apache-2.0	writing-instruct	2.0	✗	Reddit-Dirty-And-WritingPrompts
kalomaze-instruct	nothingiisreal/Kalomaze-Opus-Instruct-25k-filtered	apache-2.0	writing-instruct	2.0	✓	Kalomaze-Opus-Instruct-25k
lean-github	internlm/Lean-Github	apache-2.0	math-instruct	3.0	✗	(Wu et al., 2024)
lean-workbook	pkuAI4M/LeanWorkbook	apache-2.0	math-instruct	3.0	✗	(Ying et al., 2024)
mma	casey-martin/multilingual-mathematical-autoformalization	apache-2.0	math-instruct	3.0	✗	(Jiang et al., 2023)
lean-dojo-informal	AI4M/leandojo-informalized	-	math-instruct	3.0	✗	(Yang et al., 2023)
cpp-annotations	casey-martin/oa_cpp_annotate_gen	-	generic-instruct	1.0	✓	moyix
lean-tactics	l3lab/ntp-mathlib-instruct-st	-	math-instruct	2.0	✗	(Hu et al., 2024)
college-math	ajibawa-2023/Maths-College	apache-2.0	math	1.0	✓	ajibawa-2023/Maths-College
gradeschool-math	ajibawa-2023/Maths-Grade-School	apache-2.0	math	1.0	✓	ajibawa-2023/Maths-Grade-School
general-stories	ajibawa-2023/General-Stories-Collection	apache-2.0	synthetic-text	1.0	✓	ajibawa-2023/General-Stories-Collection
amps-mathematica	XinyaoHu/AMPS_mathematica	mit	math	1.0	✗	XinyaoHu/AMPS_mathematica
amps-khan	XinyaoHu/AMPS_khan	mit	math-instruct	1.0	✗	XinyaoHu/AMPS_khan
Magpie-300k	Magpie-Align/Magpie-Pro-MT-300K-v0.1	llama3	generic-instruct	1.0	✓	(Xu et al., 2024)
Magpie-reasoning	Magpie-Align/Magpie-Reasoning-150K	llama3	generic-instruct	1.0	✓	(Xu et al., 2024)
prox-fineweb	gair-prox/FineWeb-pro	odc-by	generic-text	1.0	✗	(Zhou et al., 2024)
prox-c4	gair-prox/c4-pro	odc-by	generic-text	1.0	✗	(Zhou et al., 2024)
prox-redpajama	gair-prox/RedPajama-pro	odc-by	generic-text	1.0	✗	(Zhou et al., 2024)
prox-open-web-math	gair-prox/open-web-math-pro	odc-by	math	1.0	✗	(Zhou et al., 2024)
together-long-data	togethercomputer/Long-Data-Collections	other	longform-text	1.0	✗	(TogetherAI, 2023)
project-gutenberg-19	emozilla/pg19	apache-2.0	longform-text	1.0	✗	(Rae et al., 2019)
mathgenie	MathGenie/MathCode-Pile	apache-2.0	math	1.0	✗	(Lu et al., 2024)
reasoning-base	KingNish/reasoning-base-20k	apache-2.0	math	1.0	✓	KingNish/reasoning-base-20k
OpenMathInstruct-2	nvidia/OpenMathInstruct-2	nvidia-license	math-instruct	1.0	✓	(Toshniwal et al., 2024a)
Txt360-DM	LLM360/TxT360	odc-by	math	1.0	✗	(Liping Tang, 2024)
Txt360-ubuntu-chat	LLM360/TxT360	odc-by	Q&A-text	1.0	✗	(Liping Tang, 2024)
markdown-arxiv	neuralwork/arxiver	cc-by-nc-sa-4.0	scientific-text	2.0	✗	neuralwork/arxiver